INTELLIGENT ROBOTICS

CHAPTER 3: REINFORCEMENT LEARNING

Learning Objectives

1、What is reinforcement learning (RL)?

2、What are dynamic programming approaches to RL?

3、What are Monte Carlo approaches to RL?

4、What are Temporal Difference approaches to RL?

5、What are Policy Gradient approaches to RL?

6、What are cost functions for deep reinforcement learning?

7、What are value and policy approximation?

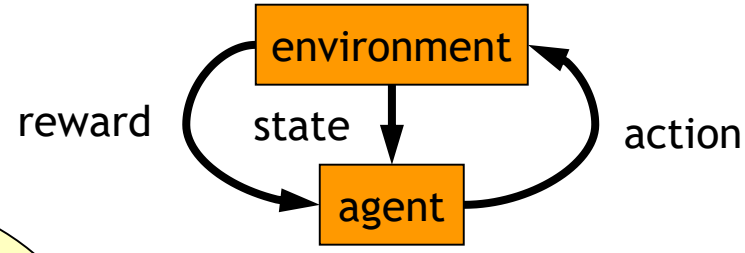
9、What is asynchronous reinforcement learning?

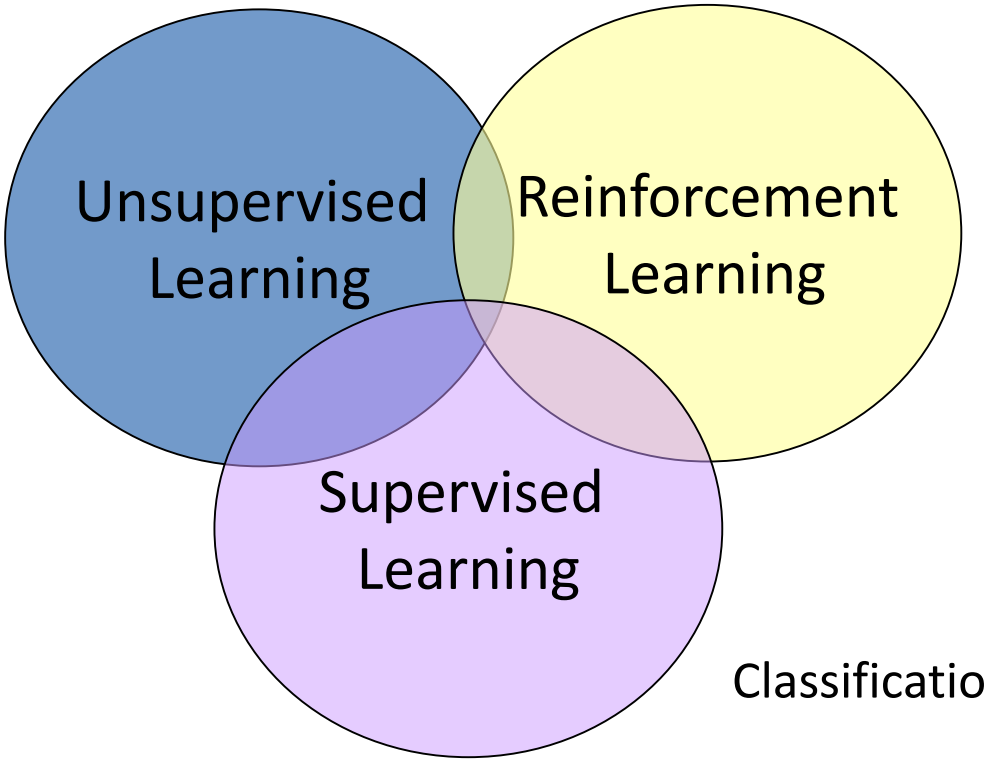
Outlines

- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

Machine Learning

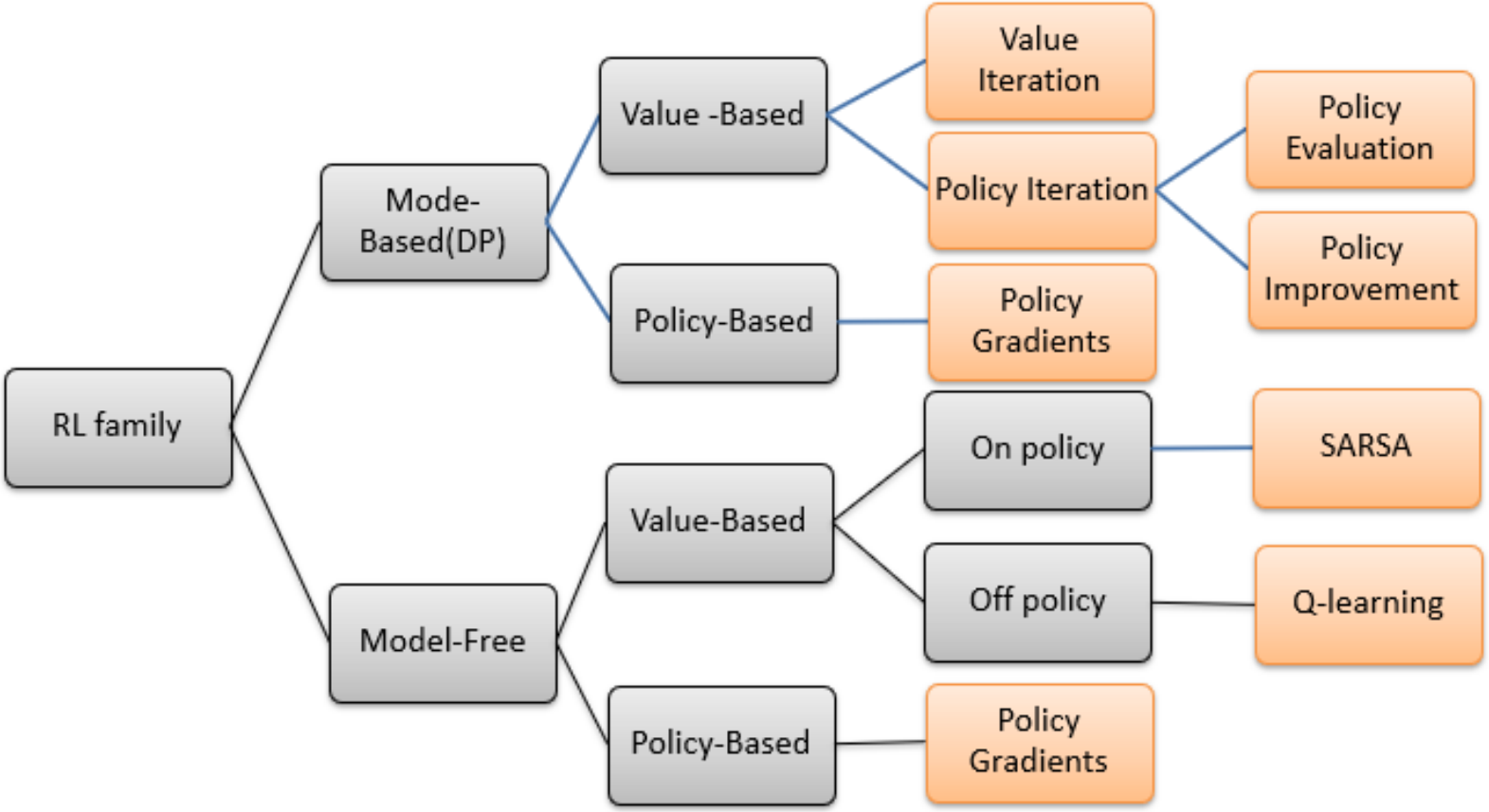
Clustering
Dimension reduction





Classification, Regression

Reinforcement Learning



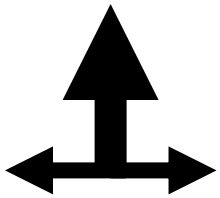
Reinforcement Learning Problem

			+1
	Obstacle		-1
START			

actions: UP, DOWN, LEFT, RIGHT

Policy

80% move UP
10% move LEFT
10% move RIGHT



reward +1 at [4,3], -1 at [4,2]

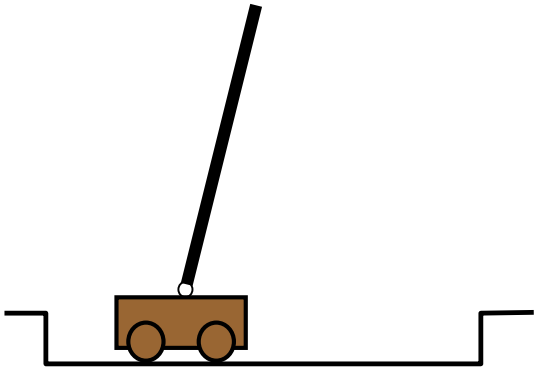
reward -0.04 for each step

what's the strategy to achieve max reward?

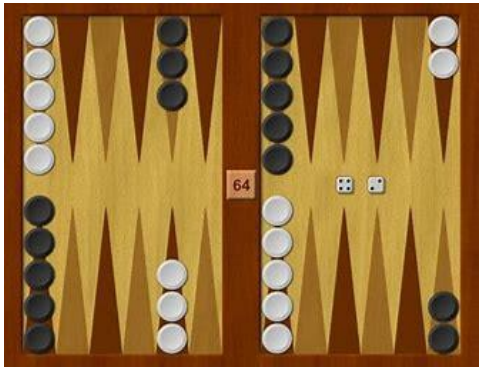
what if the actions were deterministic?

Reinforcement Learning Problems

Pole balancing



Games

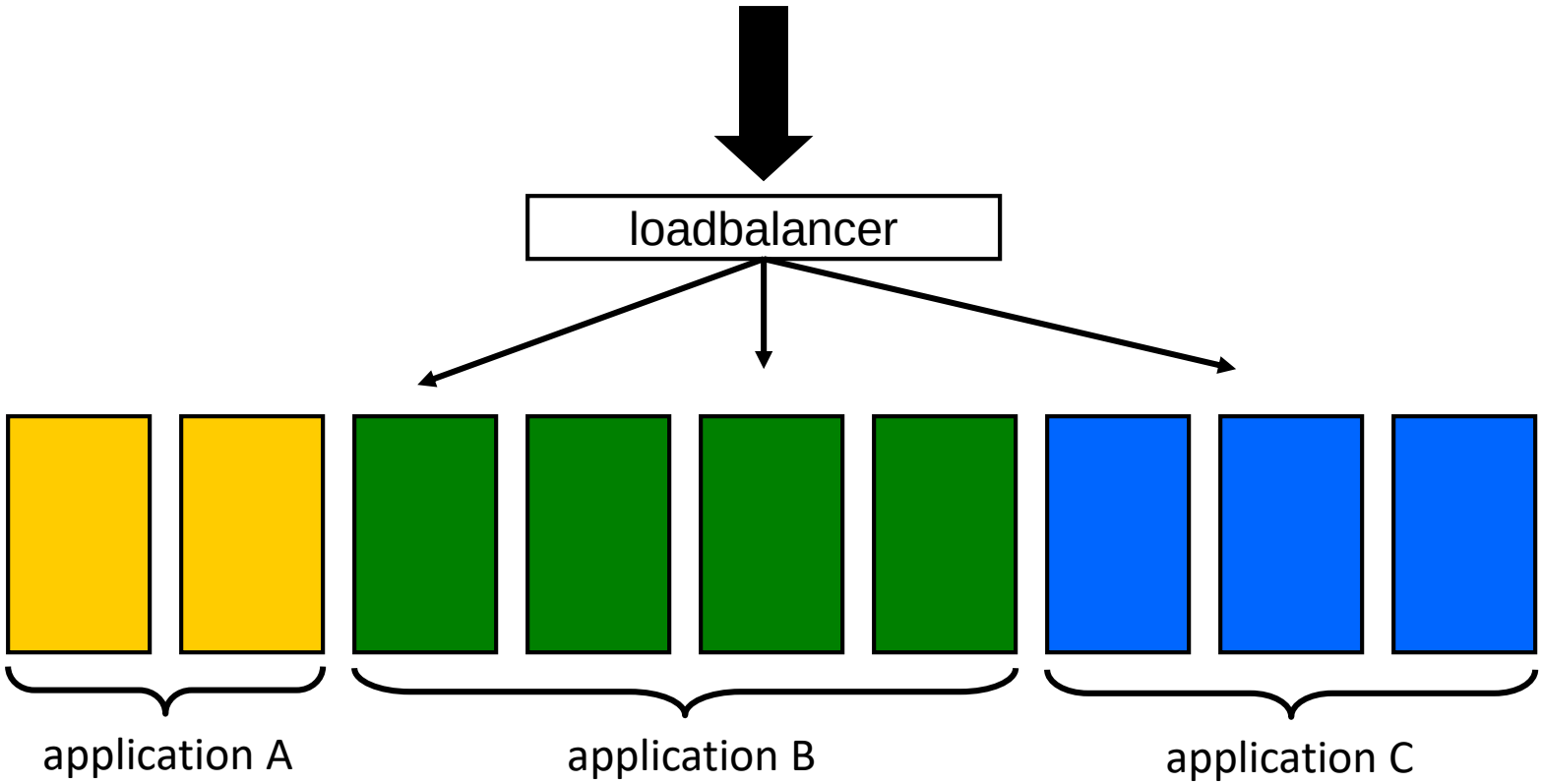


Drones

explore the environment and learn from experience

- ✓ not just blind search, try to be smart about it
- ✓ reward could be delayed

Reinforcement Learning Problem



Resource allocation among applications

Reinforcement Learning Framework

State

Action

Reward

$$x = \begin{bmatrix} A \\ J \end{bmatrix}$$

$$\boldsymbol{a} = \text{stick}$$

£££££

Policy

$$\boldsymbol{a} = \boldsymbol{\pi}(\mathbf{x})$$
$$\boldsymbol{a} \sim \boldsymbol{\pi}(\cdot \mid \mathbf{x})$$

Reward Function

$$r = R(\boldsymbol{x}_t, \ \boldsymbol{a}_t)$$
$$r \sim R(. \mid \boldsymbol{x}, \boldsymbol{a})$$

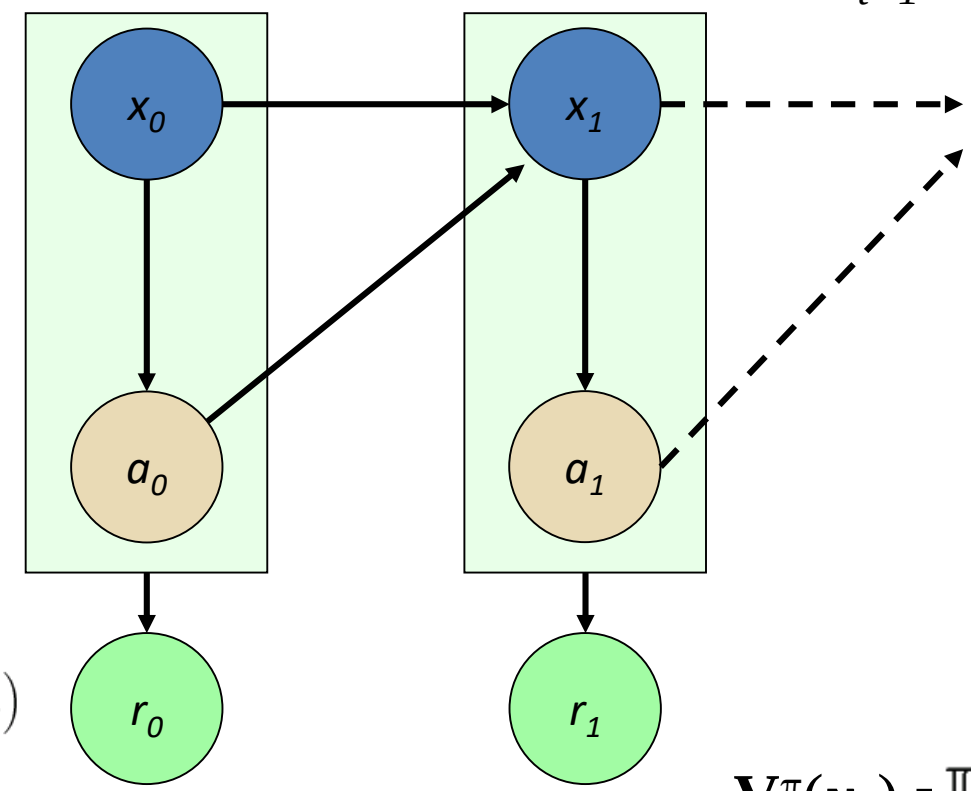
Exploration versus Exploitation

- We want a reinforcement learning agent to earn lots of reward
- The agent must prefer past actions that have been found to be effective at producing reward
- The agent must exploit what it already knows to obtain reward
- The agent must select untested actions to discover reward-producing actions
- The agent must explore actions to make better action selections in the future
- Trade-off between exploration and exploitation

Markov Decision Process (MDP)

Transition Probability

$$x_{t+1} \sim p\,(\cdot \mid \boldsymbol{x_t}, \boldsymbol{a_t})$$



Policy

$$a \sim \pi\,(\cdot \mid \boldsymbol{x})$$

Reward function

$$r \sim R(. \mid \boldsymbol{x}, \boldsymbol{a})$$

$$\mathbf{V}^{\pi}(\boldsymbol{x}_0) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t\, r_t\right]$$

Optimal Policy

Assume one optimal action per state

$$V^*(\boldsymbol{x}_0) = \max_{\boldsymbol{a}_0} \left(\underline{r(\boldsymbol{x}_0, \boldsymbol{a}_0)} + \gamma \sum_{x_1} \underline{p(\boldsymbol{x}_1 \mid \boldsymbol{x}_0, \boldsymbol{a}_0)} \, V^*(\boldsymbol{x}_1) \right)$$

Value Iteration

$a_0{}^* \sim \pi^* (\cdot | \, \boldsymbol{x}_0 \,)$

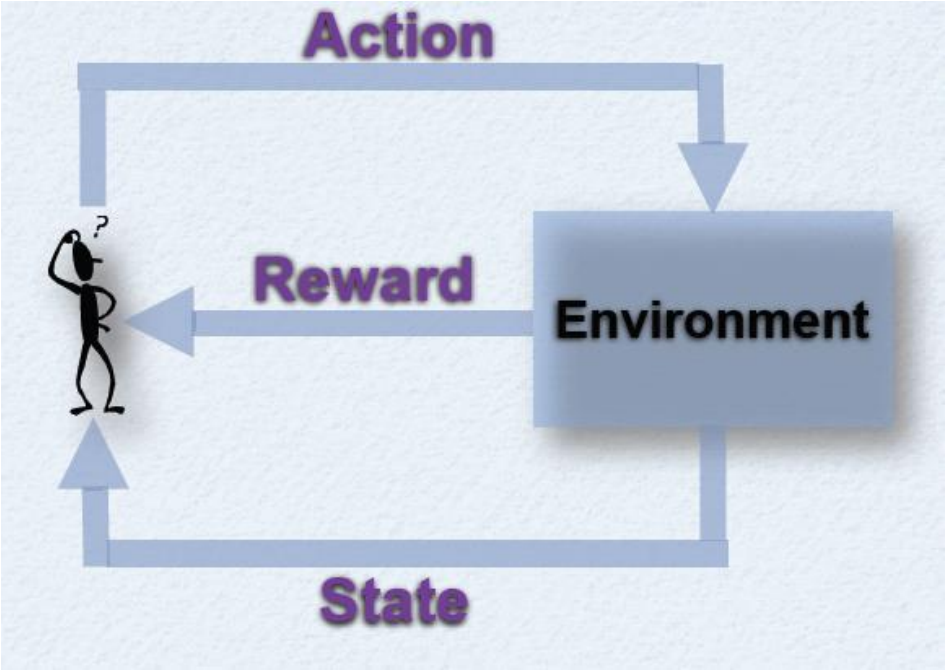
$a_1{}^* \sim \pi^* (\cdot | \, \boldsymbol{x}_1 \,)$

$a_2{}^* \sim \pi^* (\cdot | \, \boldsymbol{x}_2 \,)$

Unknown for RL

Known for MDP

Reinforcement Learning Basics



- **RL Problem:** Solve MDP when reward/transition models are unknown
- **Basic Idea:** Use samples obtained from agent's interaction with environment

Reinforcement Learning Systems

- Reinforcement learning systems have 4 main elements:
  - ✓ Policy
  - ✓ Reward signal
  - ✓ Value function
  - ✓ Optional model of the environment

Reinforcement Learning Systems

- Reinforcement learning systems have 4 main elements:
  - ✓ Policy
  - ✓ Reward
  - ✓ Value function
  - ✓ Optional model of the environment

Policy

- A policy is a mapping from the perceived states of the environment to actions to be taken when in those states
- A reinforcement learning agent uses a policy to select actions given the current environment state

Reward

- The reward defines the goal
- On each time step, the environment sends a single number called the reward to the reinforcement learning agent
- The agent's objective is to maximize the total reward that it receives over the long run
- The reward is used to alter the policy

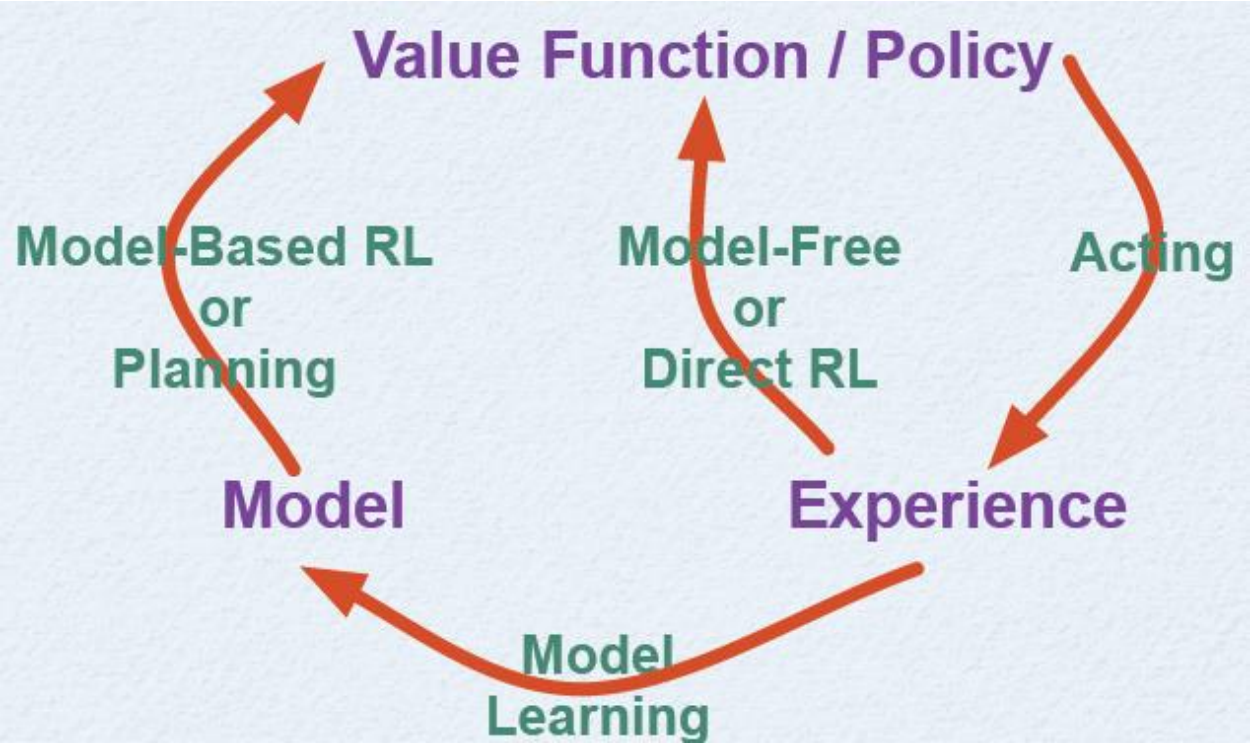
Value Function (1)

- The reward signal indicates what is good in the short run while the value function indicates what is good in the long run
- The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting in that state
- Compute the value using the states that are likely to follow the current state and the rewards available in those states
- Future rewards may be time-discounted with a factor in the interval [0, 1]

Value Function (2)

- Use the values to make and evaluate decisions
- Action choices are made based on value judgement
- Prefer actions that bring about states of highest value instead of highest reward
- Rewards are given directly by the environment
- Values must continually be re-estimated from the sequence of observations that an agent makes over its lifetime

Reinforcement Learning Models



- **Model-Based:** Learn a model of the reward/transition dynamics and derive the value function/policy
- **Model-Free:** Directly learn the value function/policy

Model-free versus Model-based

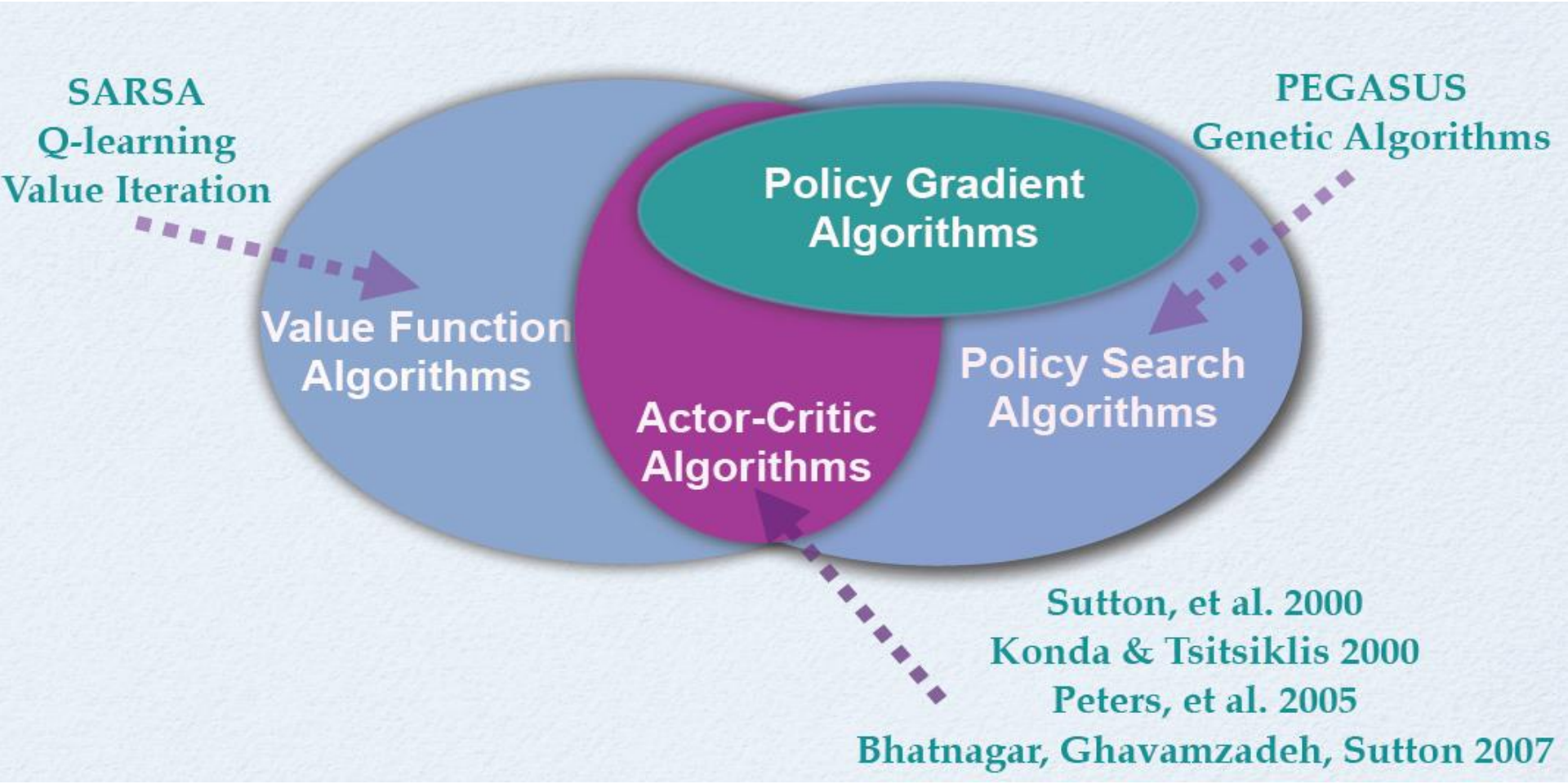
- A model of the environment allows inferences to be made about how the environment will behave
  - ✓ Example: Given a state and an action to be taken while in that state, the model could predict the next state and the next reward
- Models are used for planning, which means deciding on a course of action by considering possible future situations before they are experienced
- Model-based methods use models and planning. Think of this as modeling the dynamics $p(s' \mid s, a)$
- Model-free methods learn exclusively from trial-and-error (i.e. no modeling of the environment)

Model-Free Solutions

1. **Prediction:** Estimate V(x) or Q(x,a)

2. **Control:** Extract policy
   - On-Policy
   - Off-Policy

Reinforcement Learning Schemes



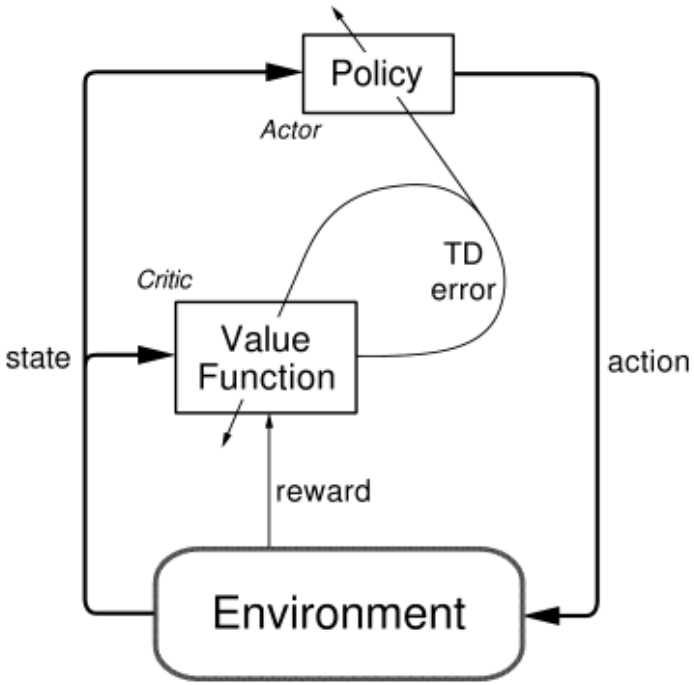
Value-Based RL Solutions

- Value Function Algorithms
  - ✓ Define a form for the value function
  - ✓ Sample state-action-reward sequence
  - ✓ Update value function
  - ✓ Extract optimal policy

SARSA, Q-learning

Actor-Critic RL Solutions

- Actor-Critic
  - ✓ Define a policy structure (*actor*)
  - ✓ Define a value function (*critic*)
  - ✓ Sample state-action-reward
  - ✓ Update both actor & critic



Policy-Based RL Solutions

- Policy Search Algorithm
  - ✓ Define a form for the policy
  - ✓ Sample state-action-reward sequence
  - ✓ Update policy

- PEGASUS

  (Policy Evaluation-of-Goodness And Search Using Scenarios)

Online - Offline

- Offline
  - ✓ Use a simulator
  - ✓ Policy fixed for each ‘episode’
  - ✓ Updates made at the end of episode

- Online
  - ✓ Directly interact with environment
  - ✓ Learning happens step-by-step

On-policy versus Off-policy

- An on-policy agent learns only about the policy that it is executing

- An off-policy agent learns about a policy or policies different from the one that it is executing

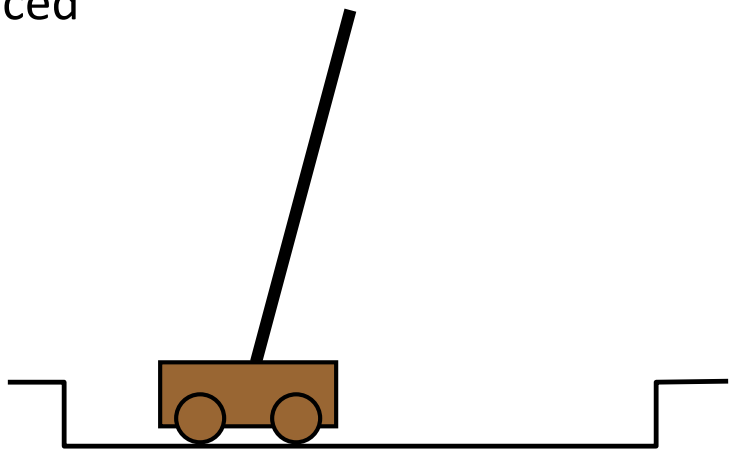
State Representation

- pole-balancing
  move car left/right to keep the pole balanced

- state representation
  position and velocity of car
  angle and angular velocity of pole

- what about *Markov property*?
  would need more info
  noise in sensors, temperature, bending of pole

- solution
  coarse discretization of state variables
    left, center, right
  totally non-Markov, but still works



Splitting and Aggregation

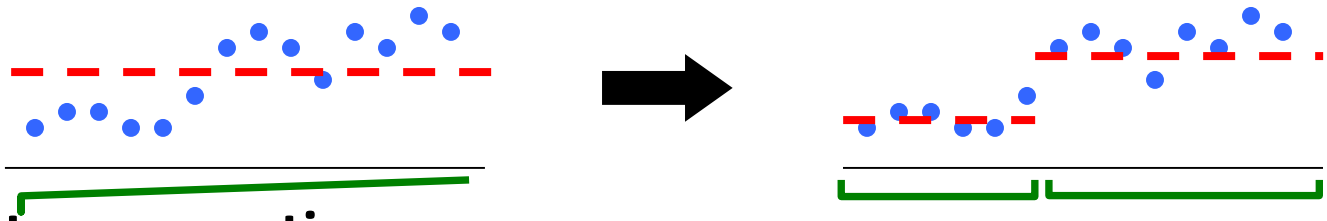
- want to discretize the state space

  learn the best discretization during training

- **splitting of state space**

  start with a single state

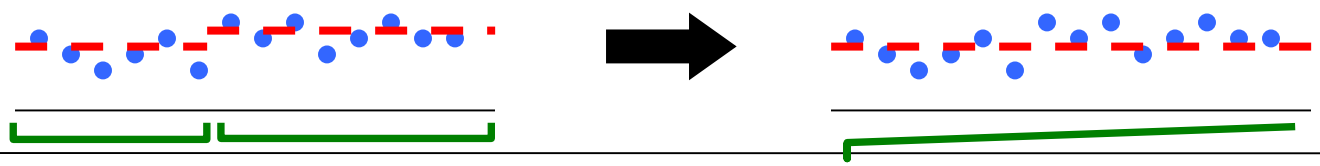
  split a state when different *parts of that state* have different values



- **state aggregation**

  start with many states

  merge states with similar values



Credit Assignment Problem

- Given a sequence of states and actions, and the final sum of time-discounted future rewards, how do we infer which actions were effective at producing lots of reward and which actions were not effective?
- How do we assign credit for the observed rewards given a sequence of actions over time?
- Every reinforcement learning algorithm must address this problem

Reward Design

- We need rewards to guide the agent to achieve its goal
  - ✓ Option 1: Hand-designed reward functions

    This is a black art
  - ✓ Option 2: Learn rewards from demonstrations
- Instead of having a human expert tune a system to achieve the desired behavior, the expert can demonstrate desired behavior and the robot can tune itself to match the demonstration

Designing Rewards

- **robot in a maze**
  episodic task, not discounted, +1 when out, 0 for each step

- **chess**
  GOOD: +1 for winning, -1 losing
  BAD: +0.25 for taking opponent's pieces
  high reward even when lose

- **rewards**
  rewards indicate what we want to accomplish
  NOT how we want to accomplish it

Inverse RL

- **shaping**
  positive reward often very "far away"
  rewards for achieving sub-goals (domain knowledge)
  also: adjust initial policy or initial value function

What is Deep Reinforcement Learning?

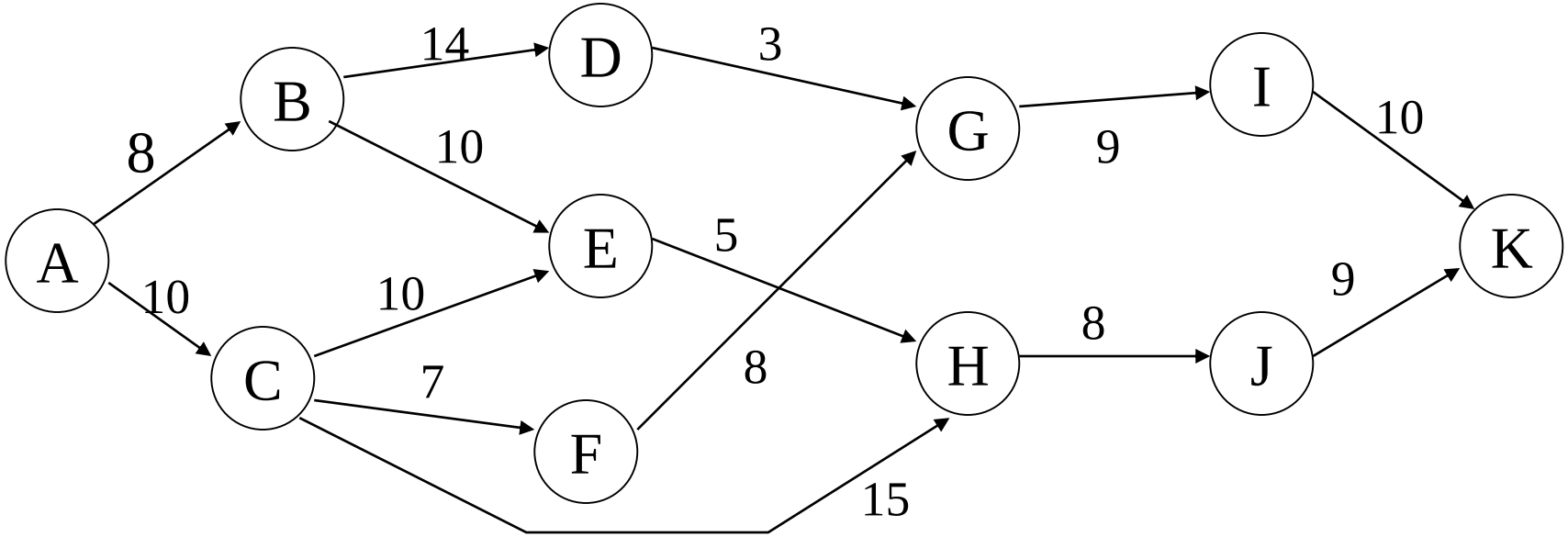
- Deep reinforcement learning is standard reinforcement learning where a deep neural network (>3 layers) is used to approximate either a policy or a value function
- Deep neural networks require lots of real/simulated interaction with the environment to learn
- Lots of trials/interactions is possible in simulated environments
- We can easily parallelize the trials/interaction in simulated environments
- We cannot do this with robotics (no simulations) because action execution takes time, accidents/failures are expensive and there are safety concerns

Outlines

- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

Dynamic Programming

Problem: How to determine the highway from A to K with the minimum total cost?



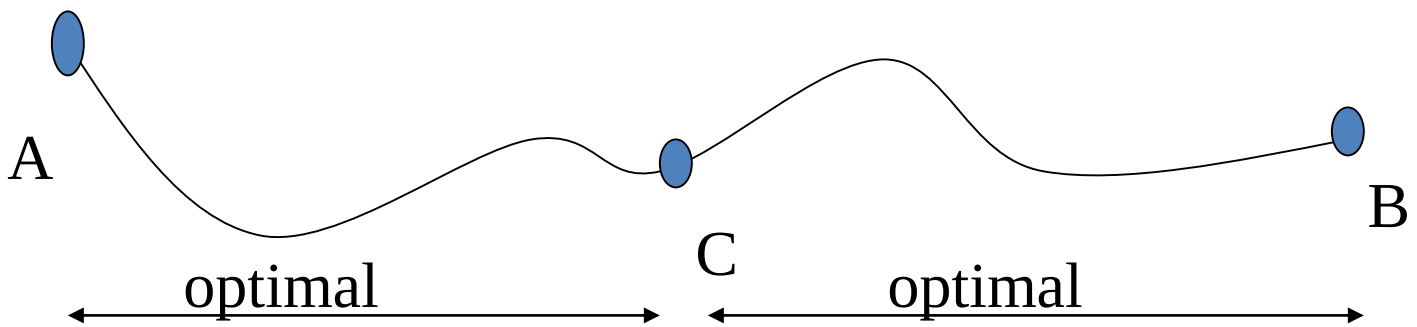
Dynamic Programming

General form:

if C belongs to an optimal path from A to B, then the sub-path A to C and C to B are also optimal

or

all sub-path of an optimal path is optimal



Dynamic Programming

- **main idea**
  - ✓ use value functions to search for good policies
  - ✓ need a perfect model of the environment

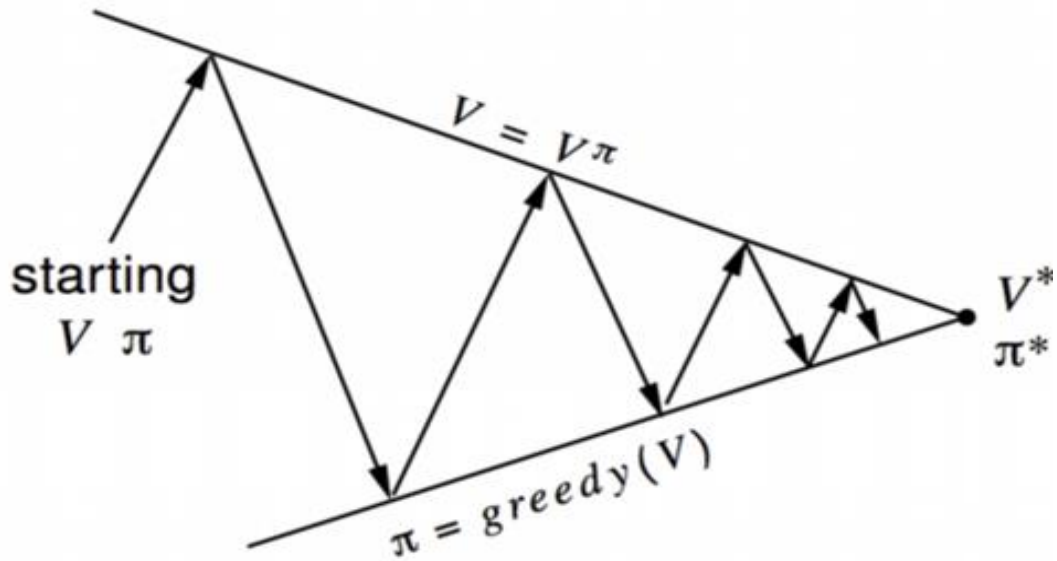
- **two main components**

  policy evaluation: compute V^π from π

  policy improvement: improve π based on V^π

  - ✓ start with an arbitrary policy
  - ✓ repeat evaluation/improvement until convergence

Dynamic Programming

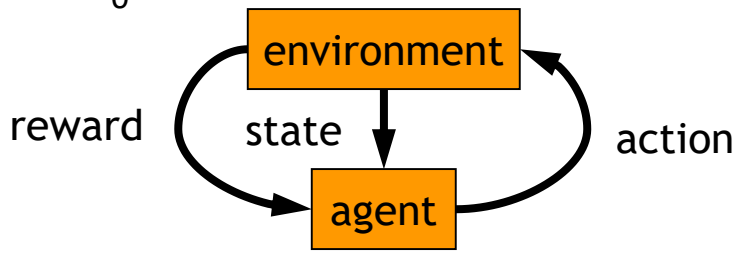


Policy evaluation Estimate v_π
Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$
Greedy policy improvement

Markov Decision Process

- **set of states**: S, **set of actions**: A, **initial state**: S_0



- **transition model**: P(s,a,s')
  P([1,1], up, [1,2]) = 0.8
- **reward function**: r(s)
  r([4,3]) = +1
- **goal**: maximize cumulative reward in the long run

- **policy**: mapping from S to A
  π(s) or π(s,a) (deterministic vs. stochastic)

- **reinforcement learning**
  - ✓ transitions and rewards usually not available
  - ✓ how to change the policy based on experience
  - ✓ how to explore the environment

Return from Rewards

- **episodic** (vs. continuing) tasks
  "game over" after N steps
  optimal policy depends on N; harder to analyze

- **additive rewards**
  $V(s_0, s_1, \ldots) = r(s_0) + r(s_1) + r(s_2) + \ldots$
  infinite value for continuing tasks

- **discounted rewards**
  $V(s_0, s_1, \ldots) = r(s_0) + \gamma * r(s_1) + \gamma^2 * r(s_2) + \ldots$
  value bounded if rewards bounded

Value Functions

- **state value function**: $V^{\pi}(s)$

  expected return when starting in *s* and following π

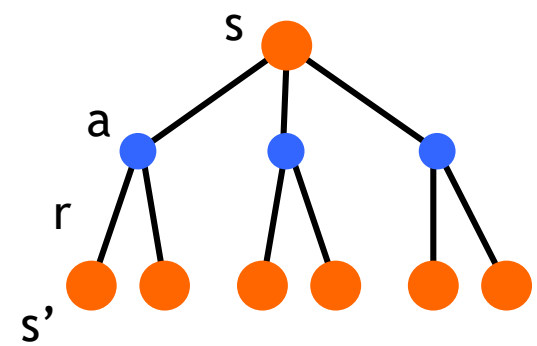
- **state-action value function**: $Q^{\pi}(s,a)$

  expected return when starting in *s*, performing *a,* and following π

- **useful for finding the optimal policy**

  can estimate from experience

  pick the best action using $Q^{\pi}(s,a)$



- **Bellman equation**

$$V^{\pi}(s) = \sum_{a} \pi(s,a) \sum_{s'} P^{a}_{ss'} \left[r^{a}_{ss'} + \gamma V^{\pi}(s') \right] = \sum_{a} \pi(s,a) Q^{\pi}(s,a)$$

Optimal Value Functions

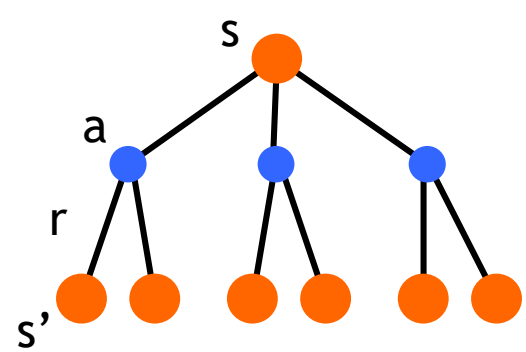
- **there's a set of *optimal* policies**
  - ✓ V^π defines partial ordering on policies
  - ✓ they share the same optimal value function $V^*(s) = \max_\pi V^\pi(s)$

- **Bellman optimality equation**
  - ✓ system of n non-linear equations
  - ✓ solve for V*(s)
  - ✓ easy to extract the optimal policy

$$V^*(s) = \max_a \overbrace{\sum_{s'} P^a_{ss'} \left[r^a_{ss'} + \gamma V^*(s') \right]}^{Q(s,a)}$$

- **having Q*(s,a) makes it even simpler**

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

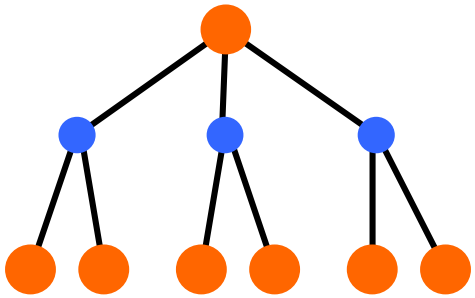


Policy Evaluation/Improvement

- **policy evaluation**: π -> V^π
  - ✓ Bellman equations define a system of n equations
  - ✓ could solve, but will use iterative version

$$V_{k+1}(s) = \sum_a \pi(s,a) \sum_{k'} P^a_{ss'} \left[r^a_{ss'} + \gamma V_k(s') \right]$$

  - ✓ start with an arbitrary value function V_0, iterate until V_k converges



- **policy improvement**: V^π -> π'

$$\pi'(s) = \arg\max_a Q^\pi(s,a)$$

$$= \arg\max_a \sum_{s'} P^a_{ss'} \left[r^a_{ss'} + \gamma V^\pi(s') \right]$$

π' either strictly better than π, or π' is optimal (if $\pi = \pi'$)

Policy/Value Iteration

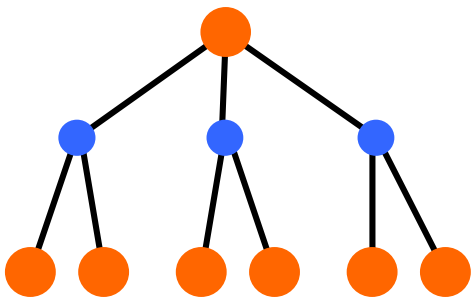
- **Policy iteration**

$$\pi_0 \rightarrow^E V^{\pi_0} \rightarrow^I \pi_1 \rightarrow^E V^{\pi_1} \rightarrow^I \ldots \rightarrow^I \pi^* \rightarrow^E V^*$$

two nested iterations; too slow

don't need to converge to V^{π_k}

just move towards it



- **Value iteration**

$$V_{k+1}(s) = \max_a \sum_{s'} P^a_{ss'} \left[r^a_{ss'} + \gamma V_k(s') \right]$$

use Bellman optimality equation as an update

converges to V*

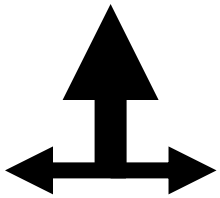
Robot in A Room

			+1
			-1
START			

actions: UP, DOWN, LEFT, RIGHT

UP

80% move UP
10% move LEFT
10% move RIGHT

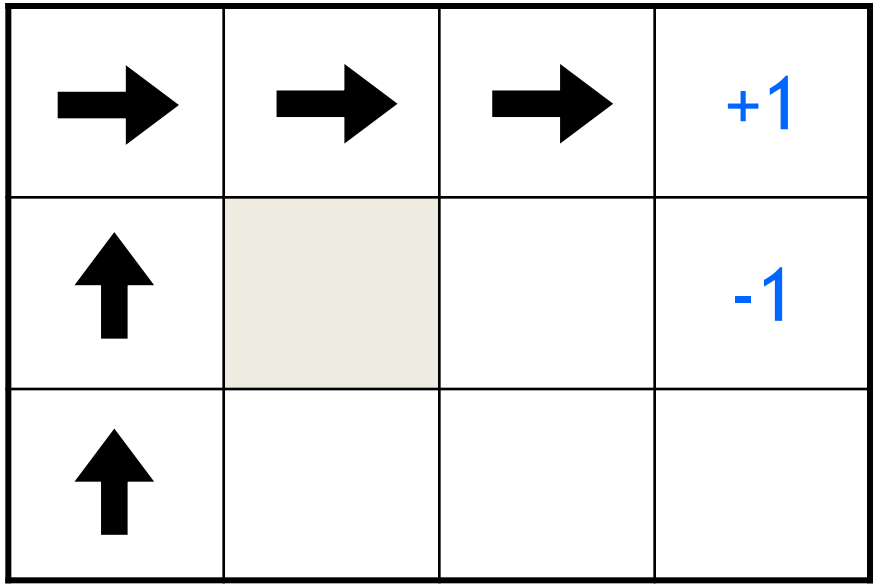


reward +1 at [4,3], -1 at [4,2]
reward -0.04 for each step

- states
- actions
- rewards

what is the solution?

Is This A Solution?



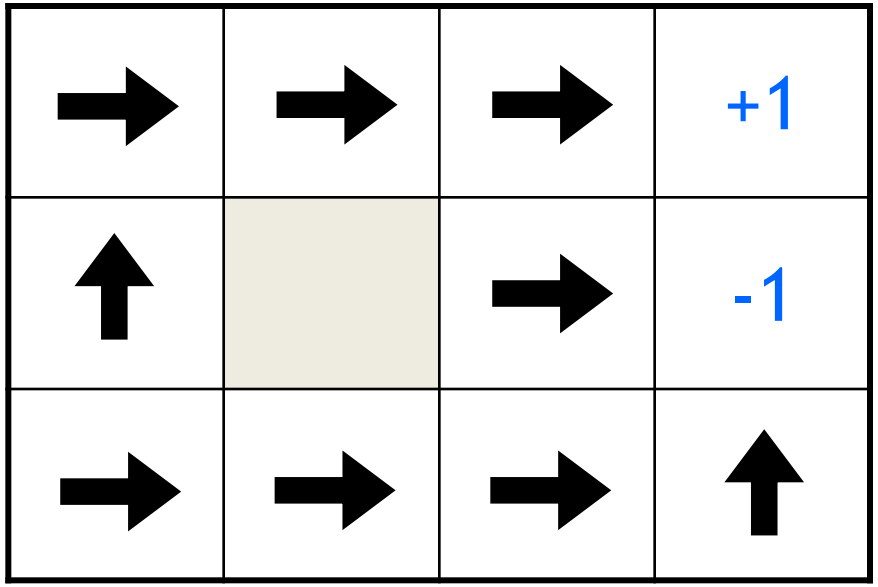
- only if actions deterministic
  not in this case (actions are stochastic)

- solution/policy
  mapping from each state to an action

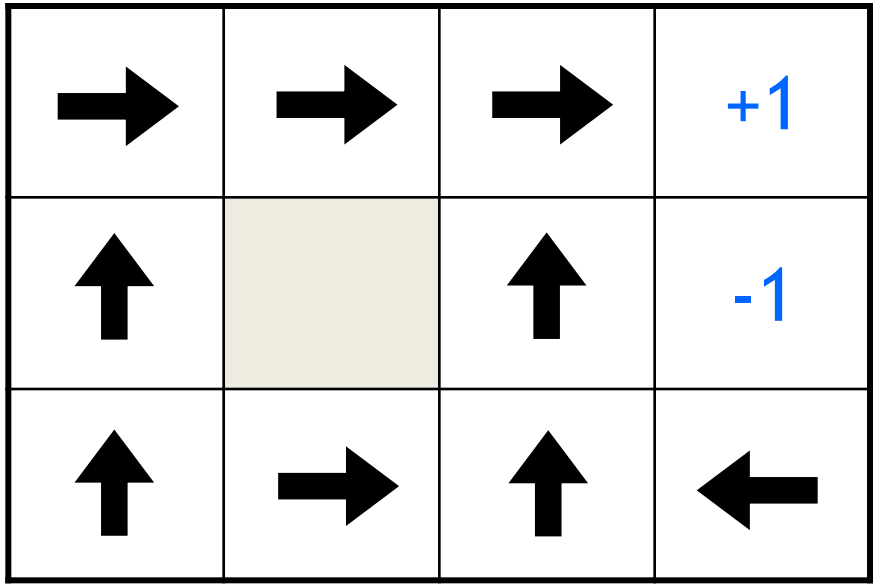
Optimal Policy

→	→	→	+1
↑		↑	-1
↑	←	←	←

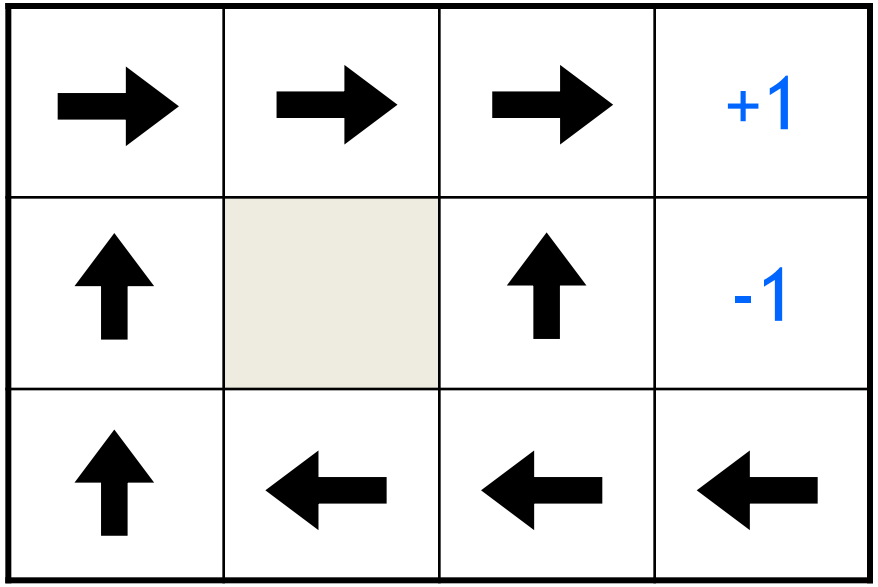
Reward for Each Step: -2



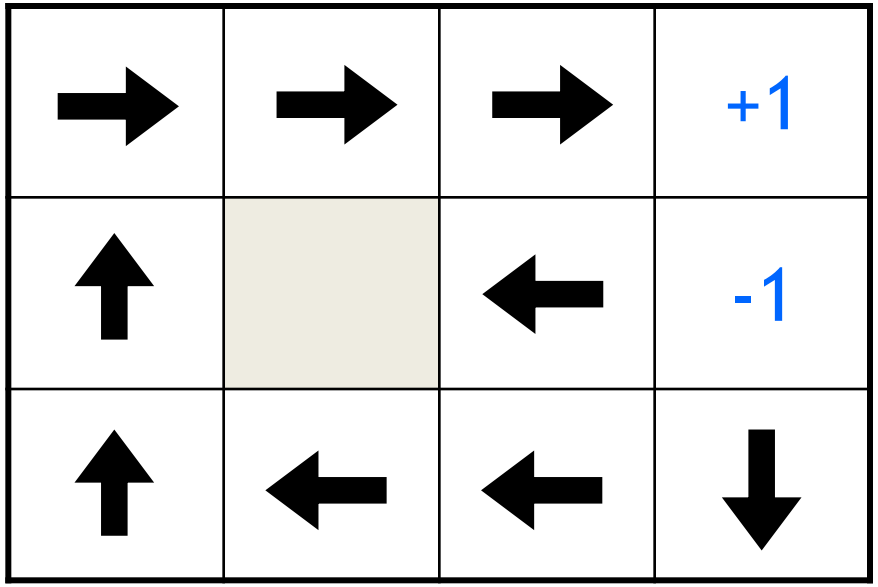
Reward for each step: -0.1



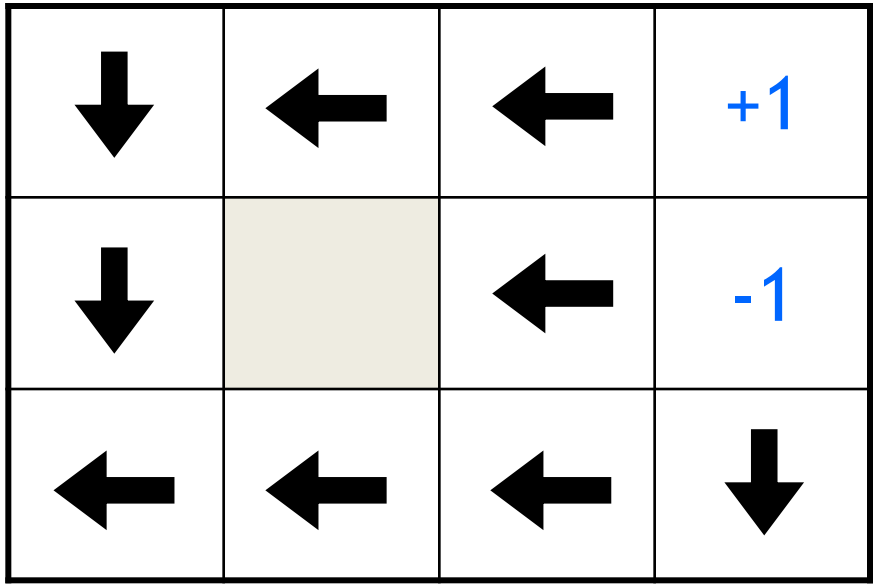
Reward for each step: -0.04



Reward for each step: -0.01



Reward for each step: +0.01



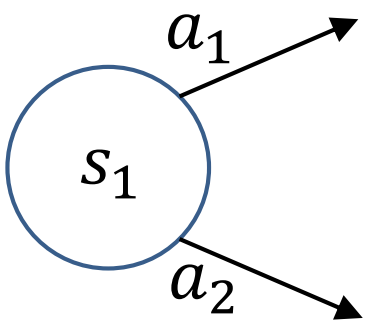
Learn Model of Environment Dynamics

- from experience

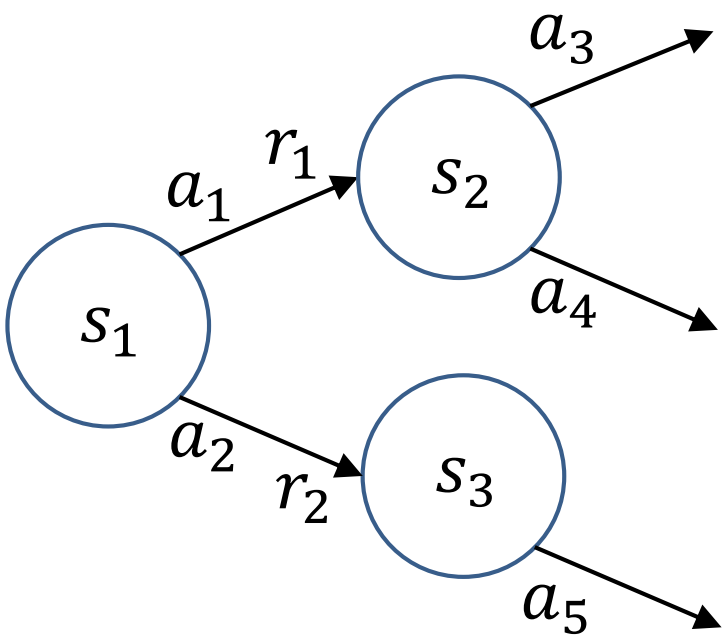
Learn Value Functions with Planning

- assuming we have the environment dynamics or a good model already

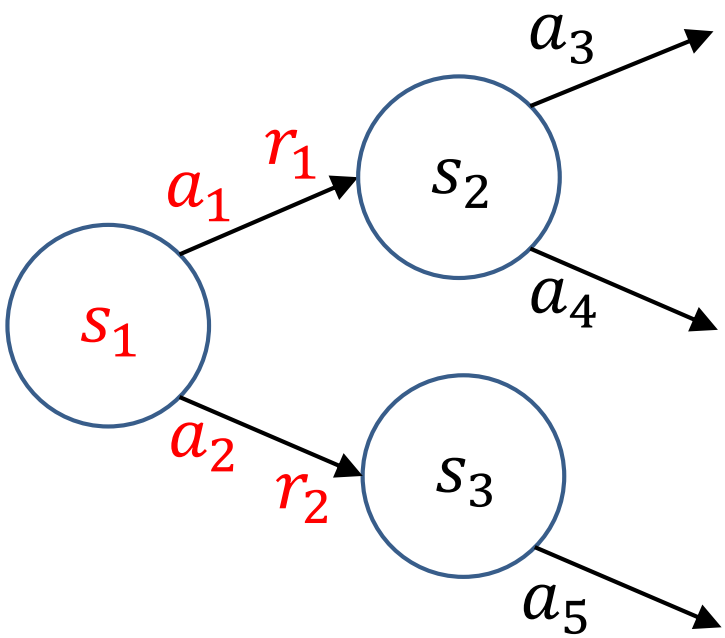
Planning



Planning

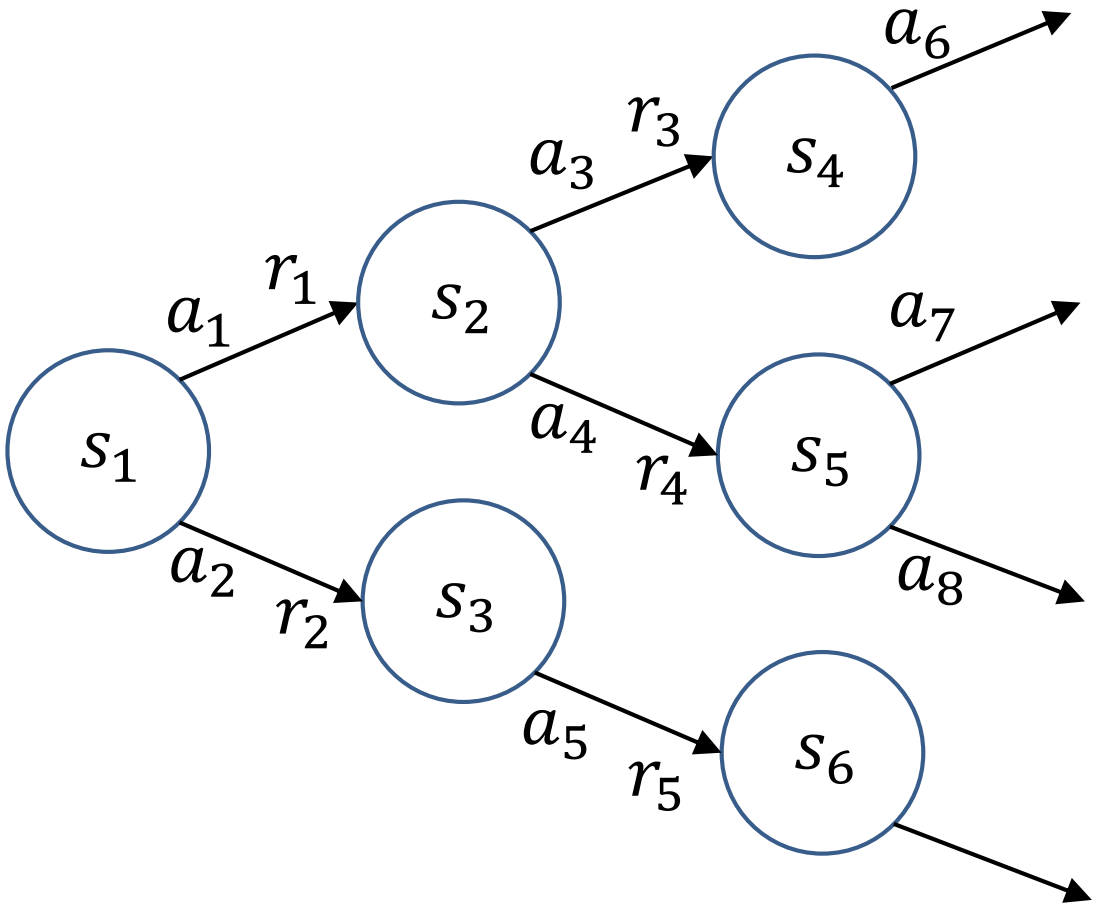


Planning

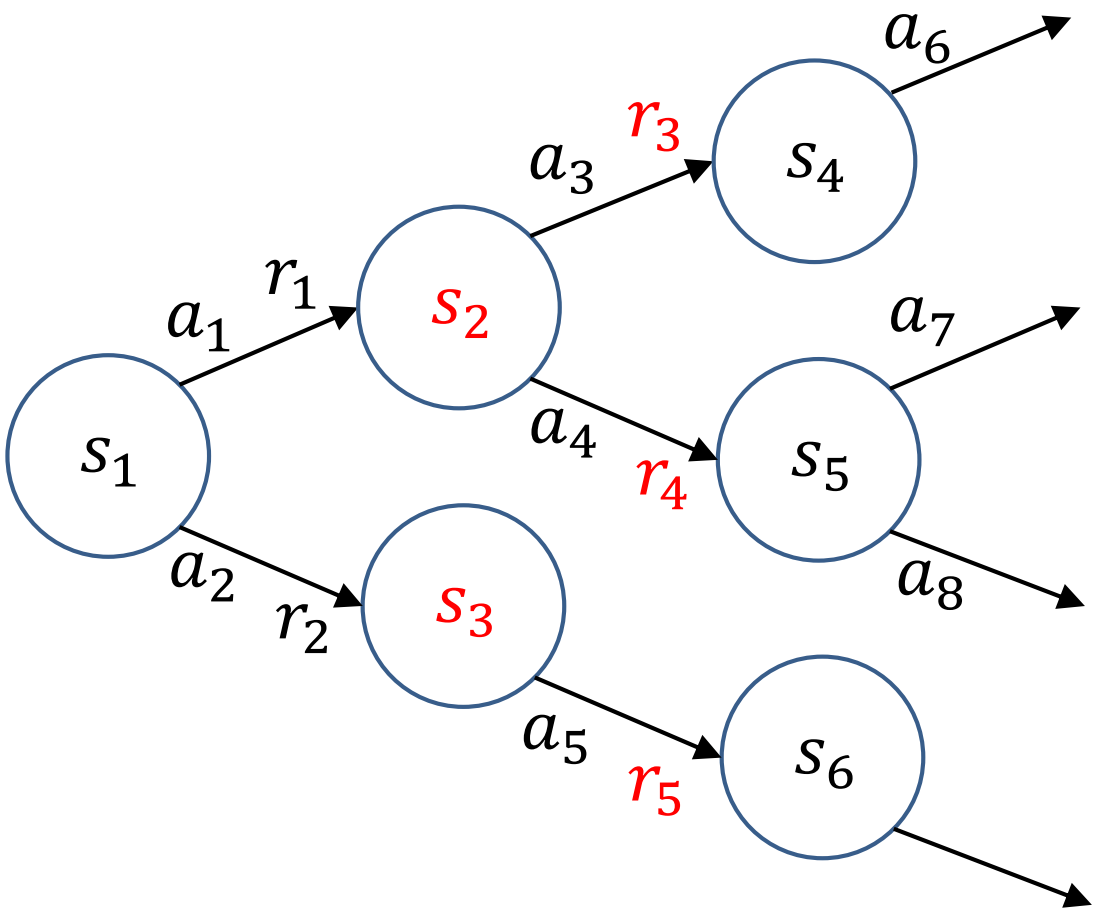


Start estimating $q(s, a)$!

Planning

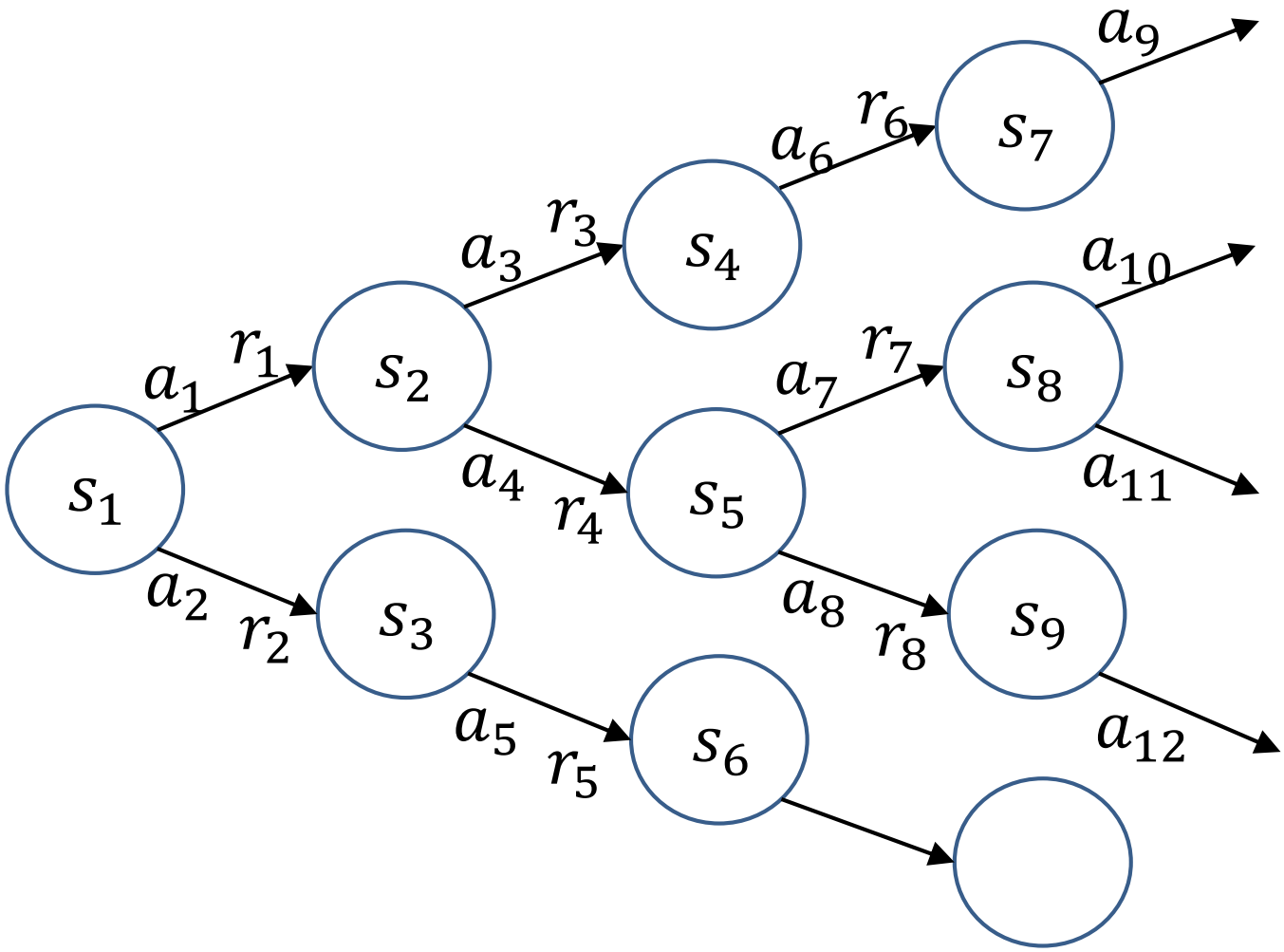


Planning

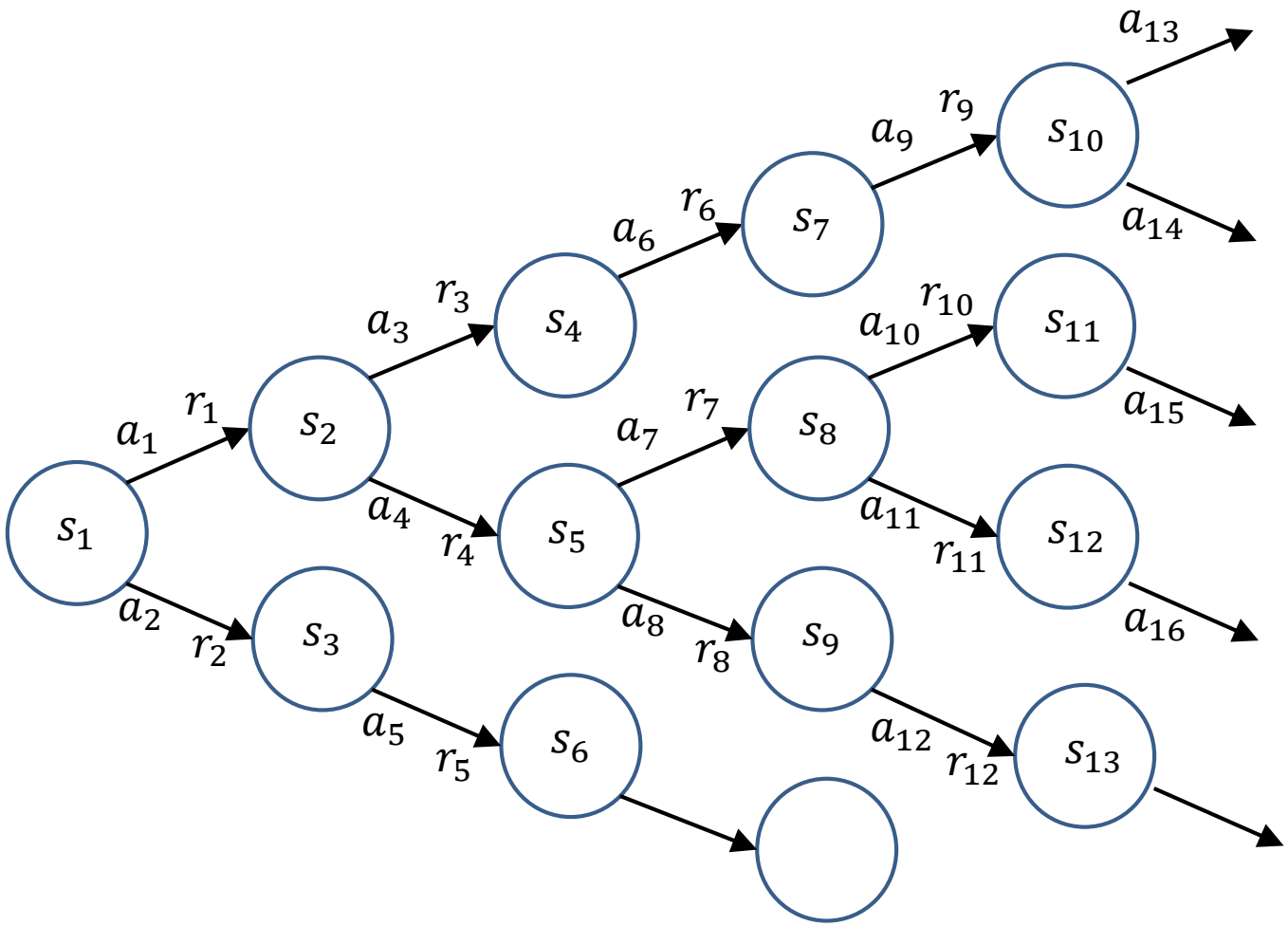


Start estimating $v(s)$!

Planning


s_1
s_2
s_3
s_4
s_5
s_6
s_7
s_8
s_9
a_1 r_1
a_2 r_2
a_3 r_3
a_4 r_4
a_5 r_5
a_6 r_6
a_7 r_7
a_8 r_8
a_9
a_{10}
a_{11}
a_{12}

Planning

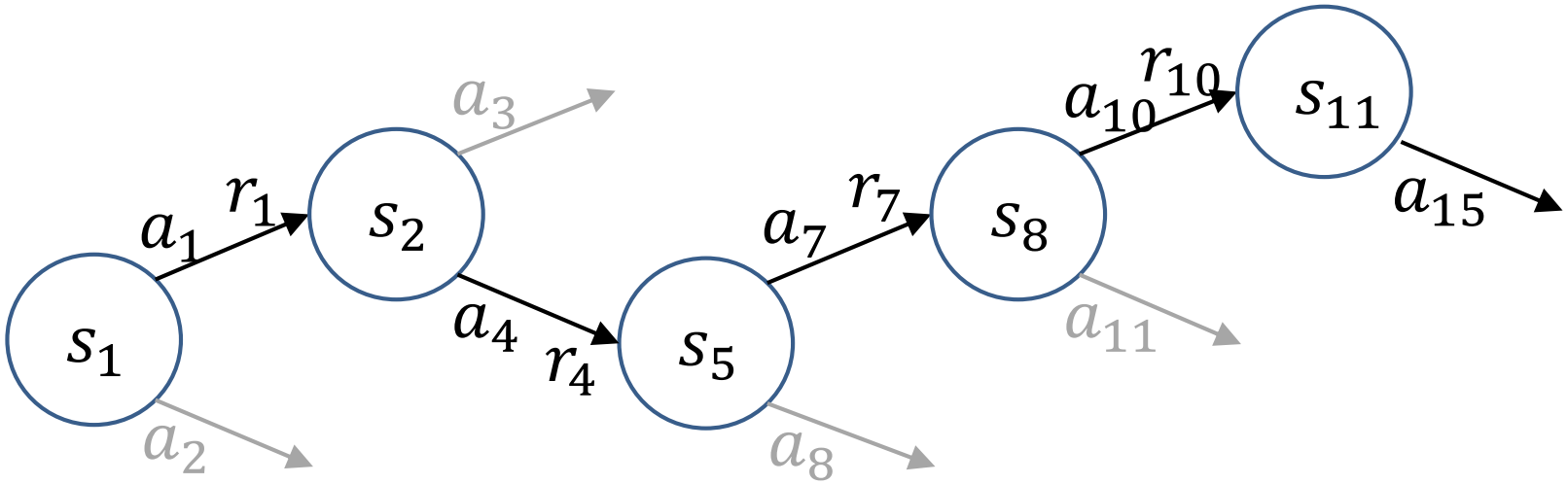

s_1
s_2
s_3
s_4
s_5
s_6
s_7
s_8
s_9
s_{10}
s_{11}
s_{12}
s_{13}
a_1 r_1
a_2 r_2
a_3 r_3
a_4 r_4
a_5 r_5
a_6 r_6
a_7 r_7
a_8 r_8
a_9 r_9
a_{10} r_{10}
a_{11} r_{11}
a_{12} r_{12}
a_{13}
a_{14}
a_{15}
a_{16}

Connections

- Dijkstra's algorithm
- A* algorithm
- minimax search

  two-player games not a problem
- Monte Carlo tree search

Roll-outs



Outlines

- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

Monte Carlo Methods

- don't need full knowledge of environment
  - ✓ just experience, or
  - ✓ simulated experience
- but similar to DP
  - ✓ policy evaluation, policy improvement
- averaging sample returns
  - ✓ defined only for episodic tasks

Monte Carlo Policy Evaluation

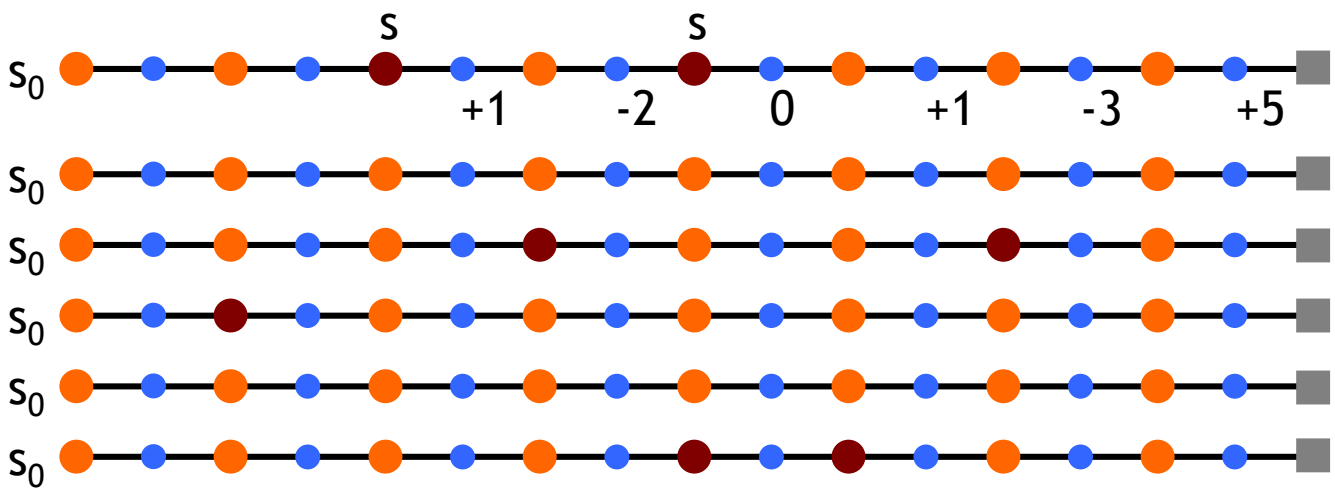
- want to estimate $V^{\pi}(s)$

  = expected return starting from s and following π

  estimate as average of observed returns in state s

- first-visit MC

  average returns following the first visit to state s



$R_1(s) = +2$

$R_2(s) = +1$

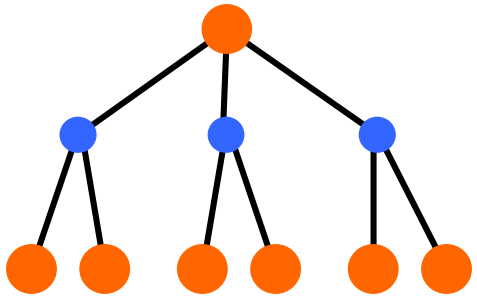
$R_3(s) = -5$

$R_4(s) = +4$

$V^{\pi}(s) \approx (2 + 1 - 5 + 4)/4 = 0.5$

Monte Carlo Control

- V^π not enough for policy improvement
  need exact model of environment



- estimate $Q^\pi(s,a)$
  $\pi'(s) = \arg\max_a Q^\pi(s,a)$

- MC control

$$\pi_0 \rightarrow^E Q^{\pi_0} \rightarrow^I \pi_1 \rightarrow^E Q^{\pi_1} \rightarrow^I \ldots \rightarrow^I \pi^* \rightarrow^E Q^*$$

- non-stationary environment

- a problem $V(s) \leftarrow V(s) + \alpha\,[R - V(s)] = (1-\alpha)V(s) + \alpha R$
  greedy policy won't explore all actions

Non-stationarity

- $v(s)$ changes over time!

Moving Average

new mean = old mean + α(new sample - old mean)

Maintaining Exploration

- deterministic/greedy policy won't explore all actions
  don't know anything about the environment at the beginning
  need to try all actions to find the optimal one

- maintain exploration
  use *soft* policies instead: $\pi(s,a)>0$ (for all s,a)

- ε-greedy policy
  with probability 1-ε perform the optimal/greedy action
  with probability ε perform a random action

  will keep exploring the environment
  slowly move it towards greedy policy: ε -> 0

Monte Carlo Predictions

	Leave car park	Get out of city	Motorway	Enter Cambridge
State				
Value	-90	-83	-55	-11
Reward	-13	-15	-61	-11
Updated	-100	-87	-72	-11

$$V(\boldsymbol{x}_t) \leftarrow V(\boldsymbol{x}_t) + \alpha(R_t - V(\boldsymbol{x}_t))$$

Simulated Experience

- 5-card draw poker

  s_0: A♣, A♦, 6♠, A♥, 2♠

  a_0: discard 6♠, 2♠

  s_1: A♣, A♦, A♥, A♠, 9♠ + dealer takes 4 cards

  return: +1 (probably)

- DP

  list all states, actions, compute P(s,a,s')

  P([A♣,A♦,6♠,A♥,2♠], [6♠,2♠], [A♠,9♠,4]) = 0.00192

- MC

  all you need are sample episodes

  let MC play against a random policy, or itself, or another algorithm

Implementation of Monte Carlo

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
- $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
- Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
- Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
  - $G \leftarrow \gamma G + R_{t+1}$
  - Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
    - Append G to $Returns(S_t, A_t)$
    - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
    - $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

Summary of Monte Carlo

- don't need a model of environment
  - ✓ averaging of sample returns
  - ✓ only for episodic tasks
- learn from sample episodes or simulated experience
- can concentrate on "important" states

  don't need a full sweep
- need to maintain exploration

  use soft policies

Outlines

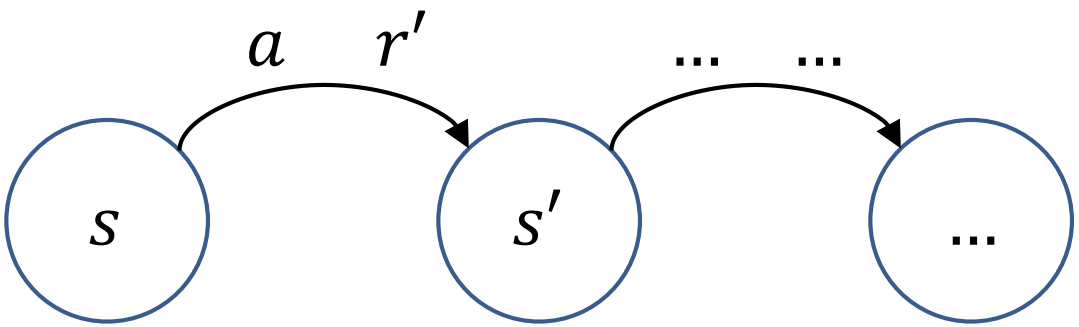
- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

What is TD learning?

- Temporal-Difference learning = TD learning
- The prediction problem is that of estimating the value function for a policy π
- The control problem is the problem of finding an optimal policy π_*
- Given some experience following a policy π, update estimate v of v_π for non-terminal states occurring in that experience
- Given current step t, TD methods wait until the next time step to update $V(S_t)$
- Learn from partial returns

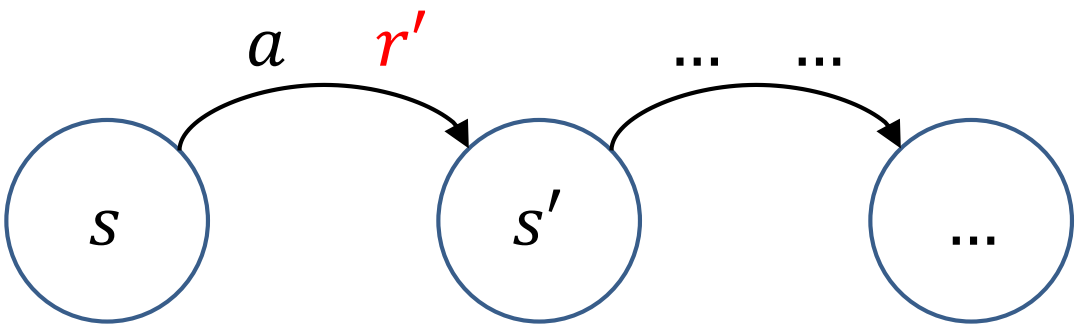
Question

What's the difference between $v(s)$ and $v(s')$?



Question

What's the difference between $v(s)$ and $v(s')$?



Bellman Equation

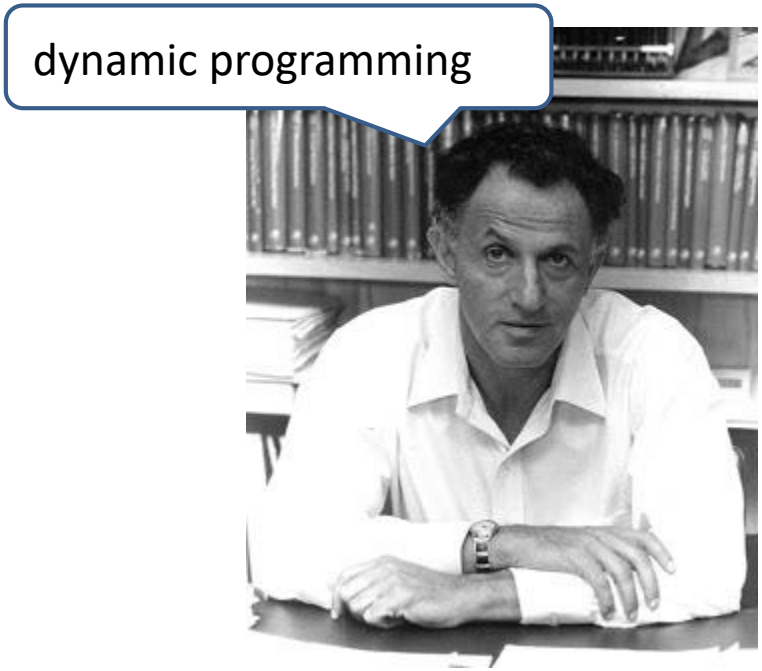
$$v(s) = r' + v(s')$$

Bellman Equation

$$v(s) = r' + v(s')$$

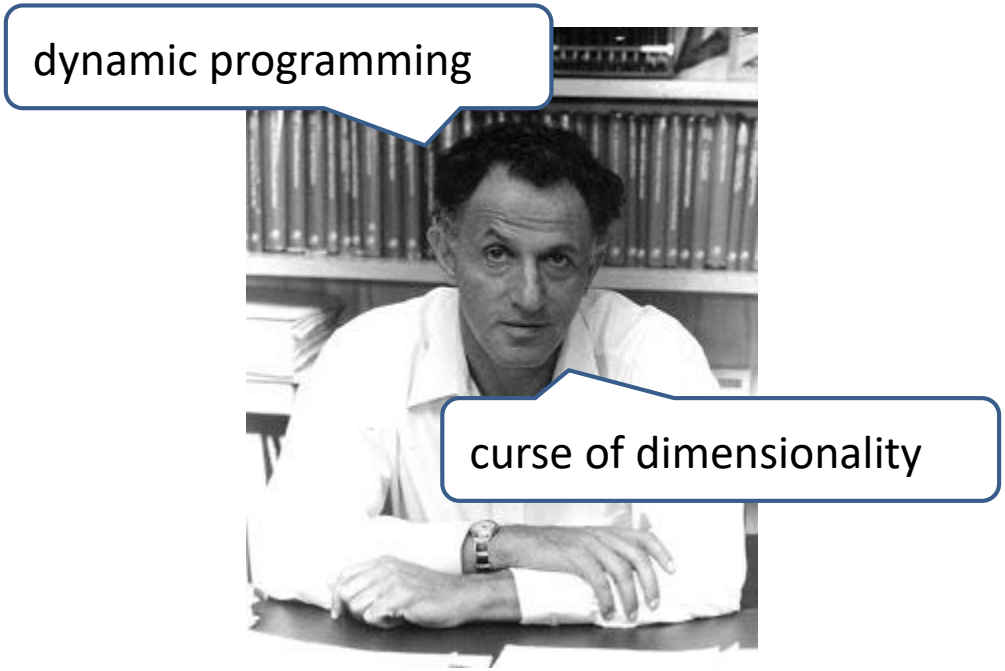
Bellman equation

$$v(s) = r' + v(s')$$



Bellman equation

$$v(s) = r' + v(s')$$



Temporal Difference (TD)

- $v(s) = r' + v(s')$
- $0 =? \; r' + v(s') - v(s)$

  "new sample": $r' + v(s')$

  "old mean": $v(s)$

Temporal Difference Learning

- combines ideas from MC and DP
  like MC: learn directly from experience (don't need a model)
  like DP: learn from values of successors
  works for continuous tasks, usually faster than MC

- constant-alpha MC:
  have to wait until the end of episode to update

$$V(s_t) \leftarrow V(s_t) + \alpha \left[R_t - V(s_t) \right]$$

target

- simplest TD
  update after every step, based on the successor

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \right]$$

Temporal Difference Predictions

	Leave car park	Get out of city	Motorway	Enter Cambridge
State				
Value	-90	-83	-55	-11
Reward	-13	-15	-61	-11
Updated	-96	-70	-72	-11

$$V(\boldsymbol{x}_t) \leftarrow V(\boldsymbol{x}_t) + \alpha(r_t + \gamma V(\boldsymbol{x}_{t+1}) - V(\boldsymbol{x}_t))$$

Advantages of TD

- Don’t need a model of reward/transitions
- Online, fully incremental
- Proved to converge given conditions on step-size
- “Usually” faster than MC methods

MC vs. TD

- observed the following 8 episodes:

A – 0, B – 0	B – 1	B – 1	B - 1
B – 1	B – 1	B – 1	B – 0

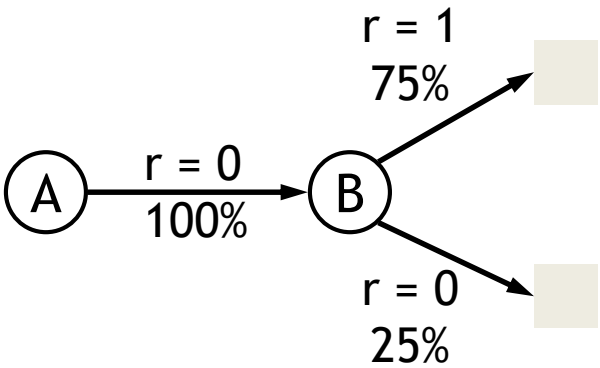
- MC and TD agree on V(B) = 3/4

- MC: V(A) = 0

  converges to values that minimize the error on training data

- TD: V(A) = 3/4

  converges to ML estimate of the Markov process



Update Rule for TD(0)

- At time t + 1, TD methods immediately form a target $R_{t+1} + \gamma V(S_{t+1})$ and make a useful update with step size α using the observed reward R_{t+1} and the estimate $V(S_{t+1})$
- The update is the step size times the difference between the target output and the actual output

Update Rule Intuition

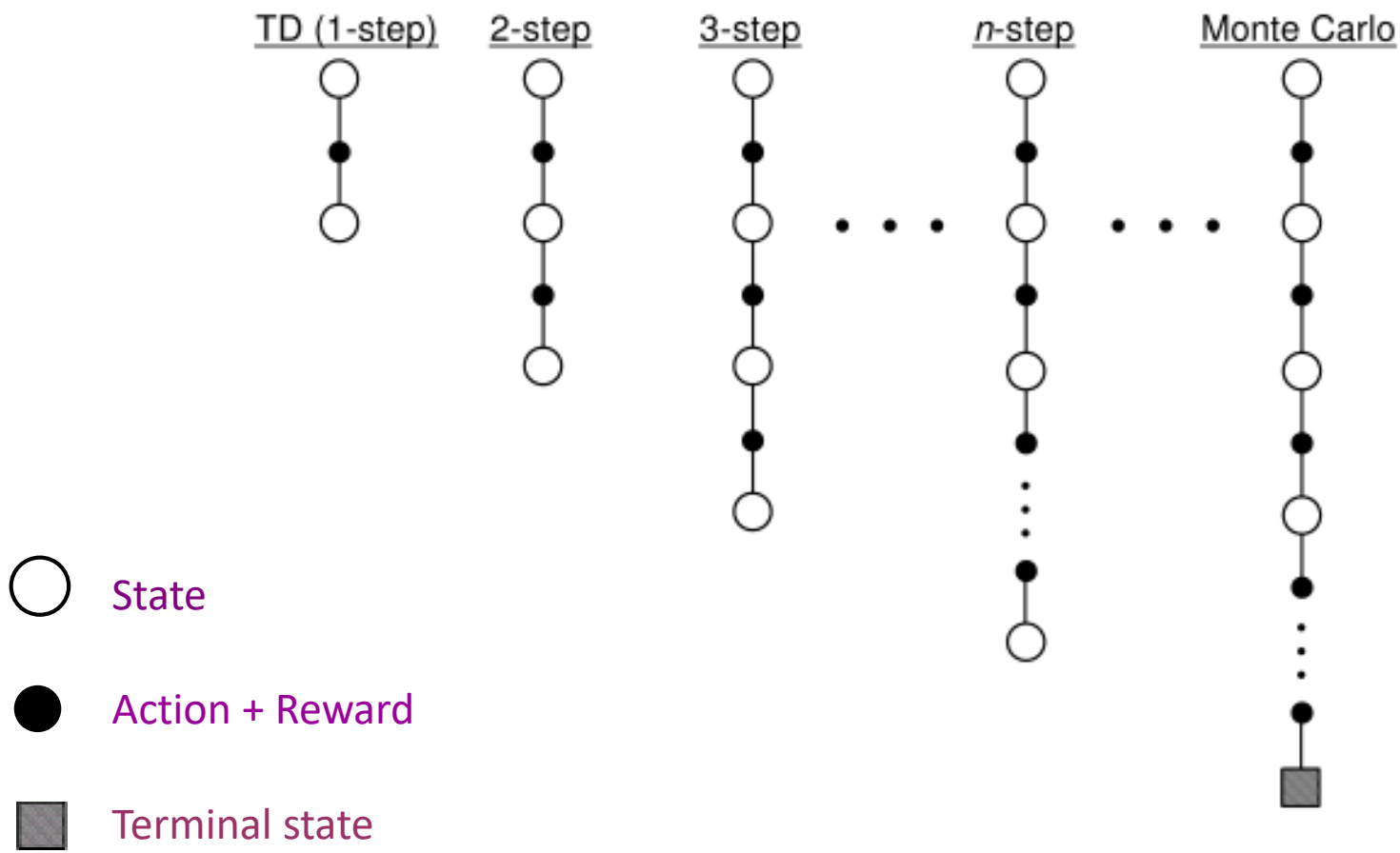
- The target output is a more accurate estimate of $V(S_t)$ given the reward R_{t+1} is known
- The actual output is our current estimate of $V(S_t)$
- We simply take one step with our current value function estimate to get a more accurate estimate of $V(S_t)$ and then perform an update to move $V(S_t)$ closer towards the more accurate estimate (i.e. temporal difference)

Tabular TD(0) Algorithm

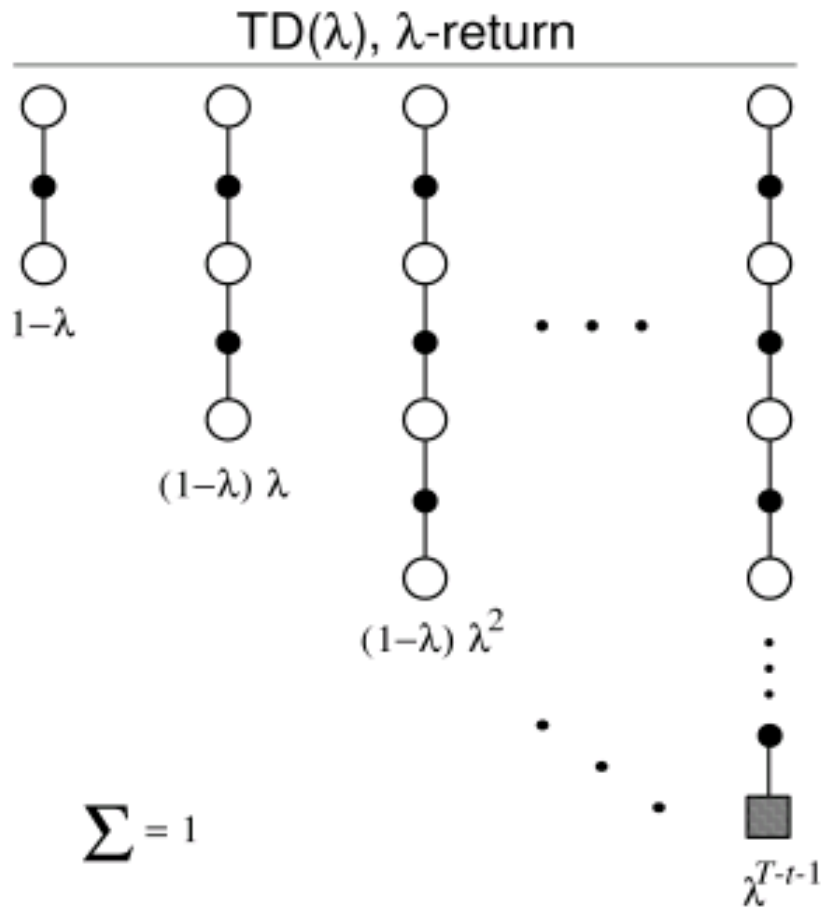
Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Initialize $V(s)$ arbitrarily (e.g., $V(s) = 0, \forall s \in \mathcal{S}^+$)
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

From TD to TD(λ)



From TD to TD(λ)



State

Action + Reward

Terminal state

SARSA & Q-learning

TD-Learning

On-Policy

Estimate value function for current policy

Off-Policy

Estimate value function for optimal policy

SARSA

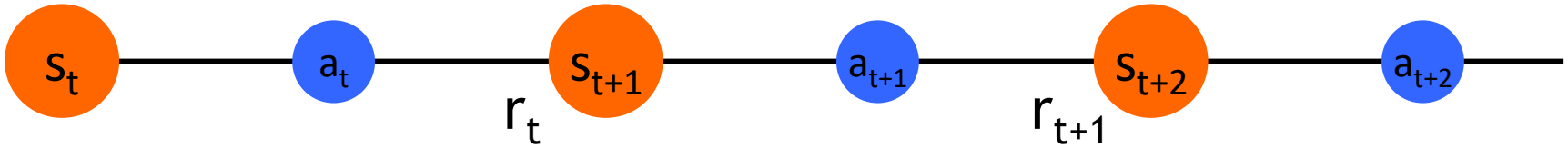
$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma Q(s',a') - Q(s,a)\right]$$

Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

SARSA

- again, need Q(s,a), not just V(s)



$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

- control

  start with a random policy

  update Q and π after each step

  again, need ε-soft policies

SARSA – On-policy TD Control

- SARSA = State-Action-Reward-State-Action
- Learn an action-value function instead of a state-value function
- q_π is the action-value function for policy π
- Q-values are the values $q_\pi(s, a)$ for s in S, a in A
- SARSA experiences are used to update Q-values
- Use TD methods for the prediction problem

SARSA Update Rule

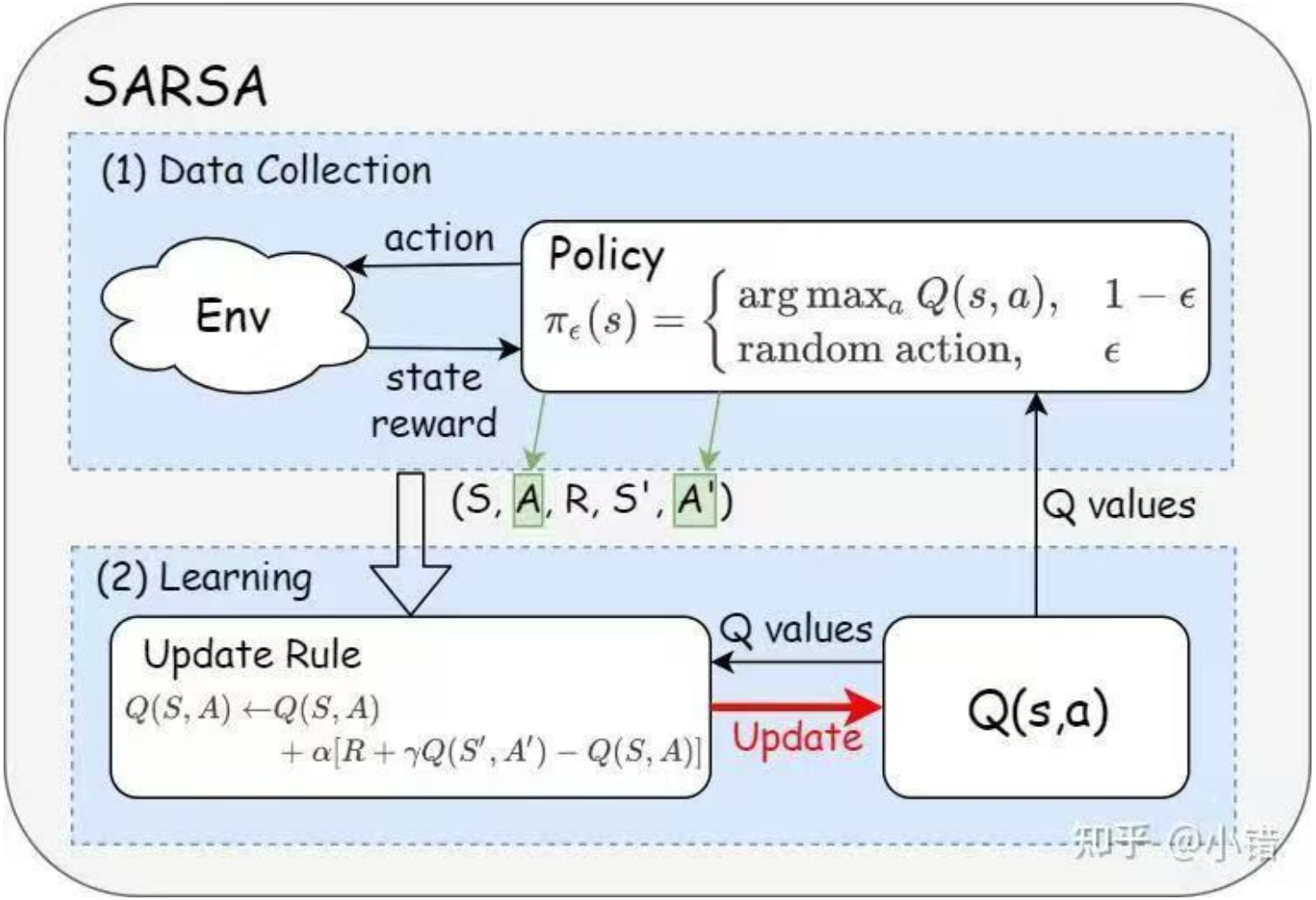
- We want to estimate $q_\pi(s, a)$ for the current policy π, and for all states s and action a
- The update rule is similar to that for TD(0) but we transition from state-action pair to state-action pair, and learn the values of state-action pairs
- The update is performed after every transition from a non-terminal state S_t
- If S_{t+1} is terminal, then $Q(S_{t+1}, A_{t+1})$ is zero
- The update rule uses $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

SARSA Algorithm

Sarsa: An on-policy TD control algorithm

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Choose A from S using policy derived from Q (e.g., ϵ-greedy)
  Repeat (for each step of episode):
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ϵ-greedy)
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma Q(S',A') - Q(S,A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

SARSA Algorithm



Q-Learning

- before: on-policy algorithms
  start with a random policy, iteratively improve
  converge to optimal

- **Q-learning**: off-policy
  use any policy to estimate Q

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

  Q directly approximates Q* (Bellman optimality eqn)
  independent of the policy being followed
  only requirement: keep updating each (s,a) pair

- **SARSA**

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

Q-learning – Off-policy TD Control

- Similar to SARSA but off-policy updates
- The learned action-value function Q directly approximates the optimal action-value function q_* independent of the policy being followed
- In update rule, choose action a that maximizes Q given S_{t+1} and use the resulting Q-value (i.e. estimated value given by optimal action-value function) plus the observed reward as the target
- This method is off-policy because we do not have a fixed policy that maps from states to actions. This is why A_{t+1} is not used in the update rule

One-step Q-learning Algorithm

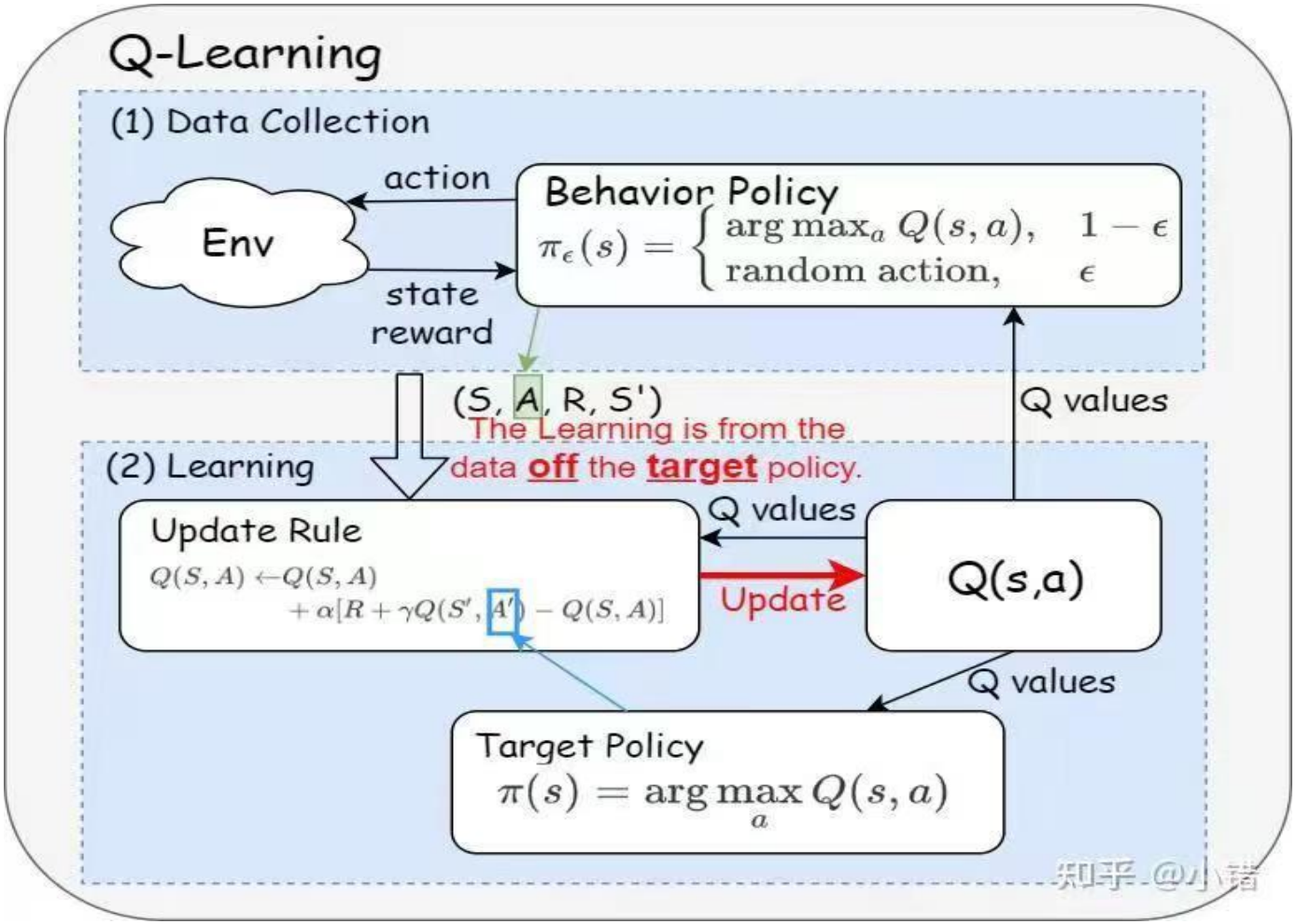
Q-learning: An off-policy TD control algorithm

Initialize $Q(s,a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\textit{terminal-state}, \cdot) = 0$
Repeat (for each episode):
  Initialize S
  Repeat (for each step of episode):
    Choose A from S using policy derived from Q (e.g., ϵ-greedy)
    Take action A, observe R, S'
    $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
    $S \leftarrow S'$
  until S is terminal

Epsilon-greedy Policy

- At each time step, the agent selects an action
- The agent follows the greedy strategy with probability 1 – epsilon
- The agent selects a random action with probability epsilon
- With Q-learning, the greedy strategy is the action a that maximizes Q given S_{t+1}

Q-Learning Algorithm



Deep Q-Networks (DQN)

- Introduced deep reinforcement learning
- It is common to use a function approximator $Q(s, a; \theta)$ to approximate the action-value function in Q-learning
- Deep Q-Networks is Q-learning with a deep neural network function approximator called the Q-network
- Discrete and finite set of actions A
- Example: Breakout has 3 actions – move left, move right, no movement
- Uses epsilon-greedy policy to select actions

Q-Networks

- Core idea: We want the neural network to learn a non-linear hierarchy of features or feature representation that gives accurate Q-value estimates
- The neural network has a separate output unit for each possible action, which gives the Q-value estimate for that action given the input state
- The neural network is trained using mini-batch stochastic gradient updates and experience replay

Function Approximation

- represent V_t as a parameterized regression function
  - ✓ linear regression, decision tree, neural net, …
  - ✓ linear regression:

$$V_t(s) = \vec{\theta}_t^T \vec{\phi}_s = \sum_{i=1}^{n} \theta_t(i)\phi_s(i)$$

- update parameters instead of entries in a table
  - ✓ better generalization

    fewer parameters and updates affect "similar" states as well

- TD update

$$V(s_t) \leftarrow V(s_t) + \alpha\left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\right]$$

$$\underbrace{V(s_t)}_{\text{f(x)}} \mapsto \underbrace{r_{t+1} + \gamma V(s_{t+1})}_{\text{y: label}}$$

  - ✓ treat as one data point for regression
  - ✓ want a method that can learn on-line (update after each step)
  - ✓ TD + NN

Experience Replay

- The state is a sequence of actions and observations $s_t = x_1, a_1, x_2, \ldots, a_{t-1}, x_t$
- Store the agent's experiences at each time step $e_t = (s_t, a_t, r_t, s_{t+1})$ in a dataset $D = e_1, \ldots, e_n$ pooled over many episodes into a replay memory
- In practice, only store the last N experience tuples in the replay memory and sample uniformly from D when performing updates

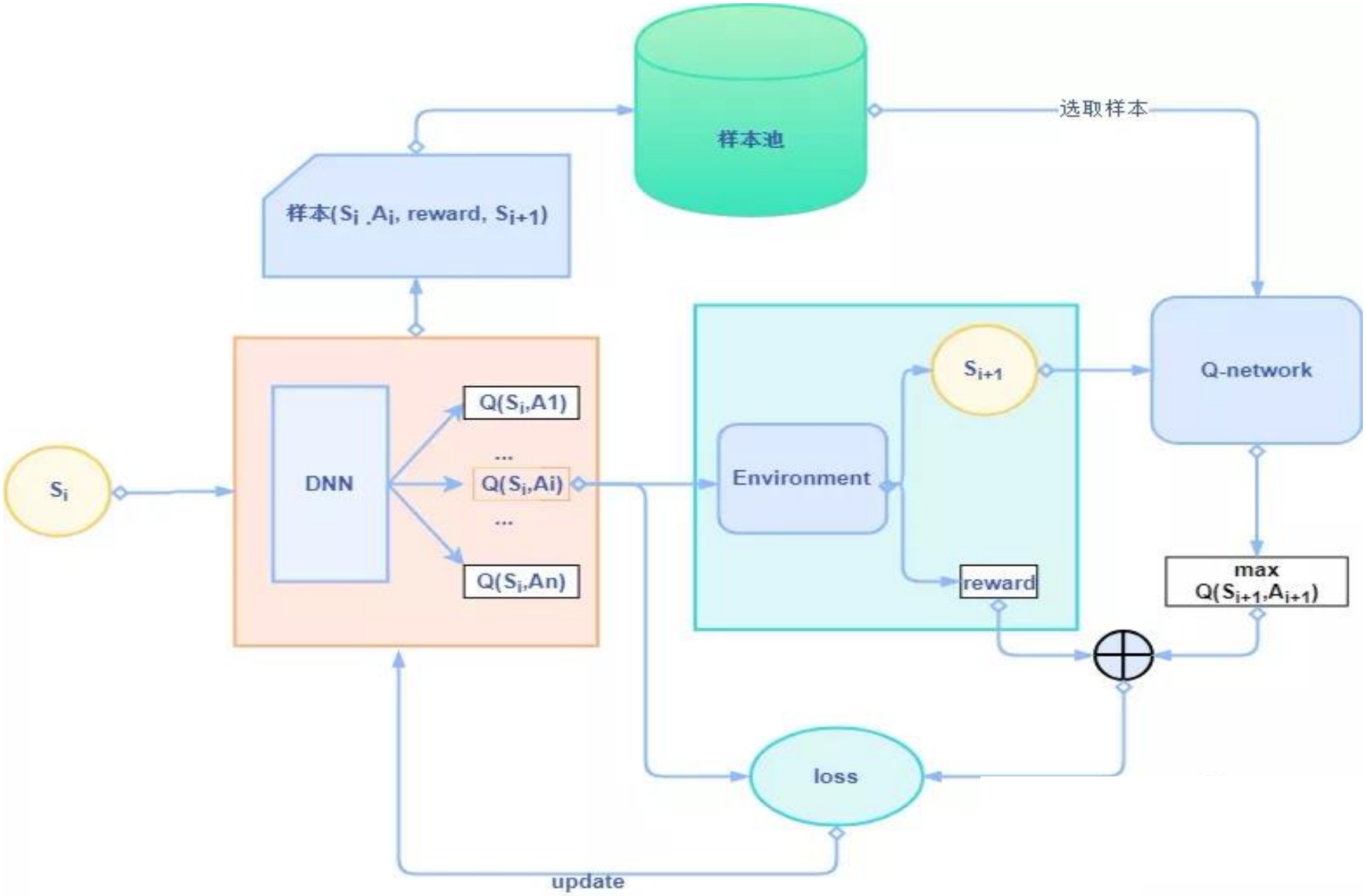
State Representation

- It is difficult to give the neural network a sequence of arbitrary length as input
- Use fixed length representation of sequence/history produced by a function $\varphi(s_t)$
- Example: The last 4 image frames in the sequence of Breakout gameplay

Q-Network Training

- Sample random mini-batch of experience tuples uniformly at random from D
- Similar to Q-learning update rule but:
  - ✓ Use mini-batch stochastic gradient updates
  - ✓ The gradient of the loss function for a given iteration with respect to the parameter θ_i is the difference between the target value and the actual value is multiplied by the gradient of the Q function approximator $Q(s, a; \theta)$ with respect to that specific parameter
- Use the gradient of the loss function to update the Q function approximator

Deep Q-Network Diagram



Loss Function Gradient Derivation

network. We refer to a neural network function approximator with weights θ as a Q-network. A Q-network can be trained by minimising a sequence of loss functions $L_i(\theta_i)$ that changes at each iteration i,

$$L_i(\theta_i) = \mathbb{E}_{s,a\sim\rho(\cdot)}\left[\left(y_i - Q(s,a;\theta_i)\right)^2\right], \tag{2}$$

where $y_i = \mathbb{E}_{s'\sim\mathcal{E}}\left[r + \gamma \max_{a'} Q(s',a';\theta_{i-1})|s,a\right]$ is the target for iteration i and $\rho(s,a)$ is a probability distribution over sequences s and actions a that we refer to as the *behaviour distribution*. The parameters from the previous iteration θ_{i-1} are held fixed when optimising the loss function $L_i(\theta_i)$. Note that the targets depend on the network weights; this is in contrast with the targets used for supervised learning, which are fixed before learning begins. Differentiating the loss function with respect to the weights we arrive at the following gradient,

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s,a\sim\rho(\cdot);s'\sim\mathcal{E}}\left[\left(r + \gamma \max_{a'} Q(s',a';\theta_{i-1}) - Q(s,a;\theta_i)\right)\nabla_{\theta_i} Q(s,a;\theta_i)\right]. \tag{3}$$

Rather than computing the full expectations in the above gradient, it is often computationally expedient to optimise the loss function by stochastic gradient descent. If the weights are updated after every time-step, and the expectations are replaced by single samples from the behaviour distribution ρ and the emulator $\mathcal{E}$ respectively, then we arrive at the familiar *Q-learning* algorithm [26].

DQN Algorithm

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation $\boxed{3}$
  end for
end for

Comments

- It was previously thought that the combination of simple online reinforcement learning algorithms with deep neural networks was fundamentally unstable
- The sequence of observed data (states) encountered by an online reinforcement learning agent is non-stationary and online updates are strongly correlated
- The technique of DQN is stable because it stores the agent's data in experience replay memory so that it can be randomly sampled from different time-steps
- Aggregating over memory reduces non-stationarity and decorrelates updates but limits methods to off-policy reinforcement learning algorithms
- Experience replay updates use more memory and computation per real interaction than online updates, and require off-policy learning algorithms that can update from data generated by an older policy

Outlines

- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

What are Policy Gradient Methods?

- Before: Learn the values of actions and then select actions based on their estimated action-values. The policy was generated directly from the value function
- We want to learn a parameterized policy that can select actions without consulting a value function. The parameters of the policy are called policy weights
- A value function may be used to learn the policy weights but this is not required for action selection
- Policy gradient methods are methods for learning the policy weights using the gradient of some performance measure with respect to the policy weights
- Policy gradient methods seek to maximize performance and so the policy weights are updated using gradient ascent

Policy-based Reinforcement Learning

- Search directly for the optimal policy π^*
- Can use any parametric supervised machine learning model to learn policies $\pi(a \mid s; \boldsymbol{\theta})$ where $\boldsymbol{\theta}$ represents the learned parameters
- Recall that the optimal policy is the policy that achieves maximum future return

Notation

- Policy weight vector $\boldsymbol{\theta}$
- The policy is $\pi(a \mid s, \boldsymbol{\theta})$, which represents the probability that action a is taken in state s with policy weight vector $\boldsymbol{\theta}$
- If using learned value functions, the value function's weight vector is $\mathbf{w}$
- Performance measure $\eta(\boldsymbol{\theta})$
- Episodic case: $\eta(\boldsymbol{\theta}) = v_\pi(s_0)$
- Performance is defined as the value of the start state under the parameterized policy

Policy Approximation

- represent π_t as a parameterized classification function
  - ✓ softmax regression, neural net, …
  - ✓ cost function: $L(\theta) = -\sum \log p(x)\, f(x) = -\sum \log \pi(A|S,\theta\,)\, f(S,A)$

    reward

- update parameters instead of entries in a table
  - ✓ better generalization

    fewer parameters and updates affect "similar" states-actions as well

- Policy gradient for update

$$g(\theta) = \nabla_\theta L(\theta) = -\sum \nabla_\theta \log \pi(A|S,\theta\,)\, f(S,A)$$

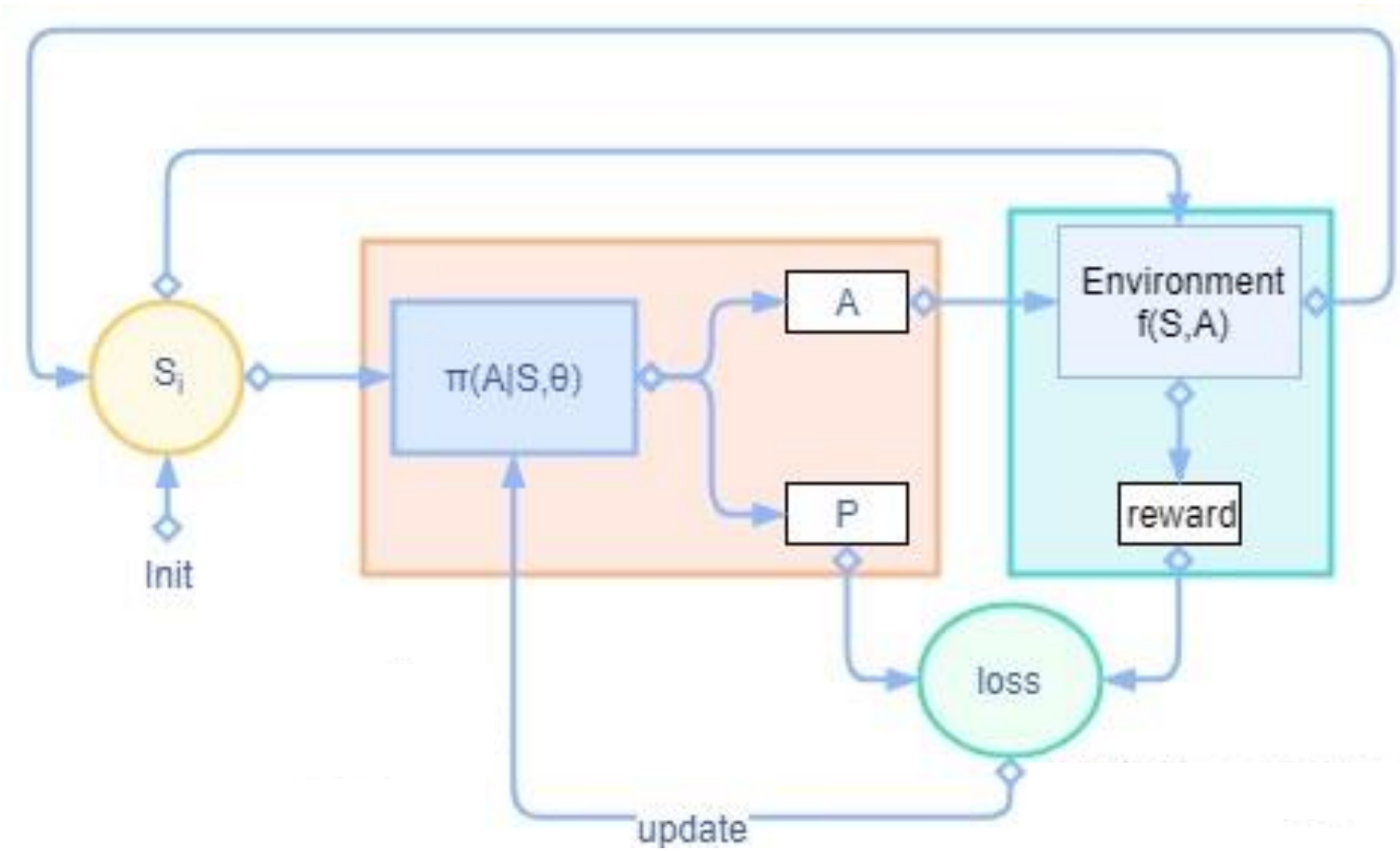
  - ✓ treat as one episode for classification
  - ✓ want a method that can learn off-line (update after each episode)
  - ✓ Monte Carlo + NN

Policy Approximation

- The policy can be parameterized in any way provided $\pi(a \mid s, \boldsymbol{\theta})$ is differentiable with respect to its weights
- To ensure exploration, we generally require that the policy never becomes deterministic and so $\pi(a \mid s, \boldsymbol{\theta})$ is in interval (0, 1)
- Assume discrete and finite set of actions
- Form parameterized numerical preferences $h(s, a, \boldsymbol{\theta})$ for each state-action pair
- Can use a deep neural network as the policy approximator
- Use softmax so that the most preferred actions in each state are given the highest probability of being selected

$$\pi(a|s,\boldsymbol{\theta}) \doteq \frac{\exp(h(s,a,\boldsymbol{\theta}))}{\sum_b \exp(h(s,b,\boldsymbol{\theta})}$$

Policy Gradient Diagram



Types of Policy Gradient Methods

- Finite Difference Policy Gradient
- Monte Carlo Policy Gradient
- Actor-Critic Policy Gradient

Finite Difference Policy Gradient

- Compute the partial derivative of the performance measure $\eta(\boldsymbol{\theta})$ with respect to θ_k using finite difference gradient approximation
- Compute performance measure $\eta(\boldsymbol{\theta})$
- Perturb $\boldsymbol{\theta}$ by small positive amount epsilon in the kth dimension and compute performance measure $\eta(\boldsymbol{\theta} + u_k\varepsilon)$ where u_k is unit vector that is 1 in kth component and 0 elsewhere
- The partial derivative is approximated by

  $$(\eta(\boldsymbol{\theta} + u_k\varepsilon) - \eta(\boldsymbol{\theta})) / \varepsilon$$

  $$\theta_k \leftarrow \theta_k + \alpha (\eta(\boldsymbol{\theta} + u_k\varepsilon) - \eta(\boldsymbol{\theta})) / \varepsilon$$
- Perform gradient ascent using the partial derivative
- Simple, noisy and inefficient procedure that is sometimes effective
- This procedure works for arbitrary policies (can be non-differentiable)

REINFORCE: Monte Carlo Policy Gradient

- We want a quantity that we can sample on each time step whose expectation is equal to the gradient
- We can then perform stochastic gradient ascent with this quantity

$$\boldsymbol{\theta}_{t+1} \doteq \boldsymbol{\theta}_t + \alpha \gamma^t G_t \frac{\nabla_{\boldsymbol{\theta}} \pi(A_t|S_t, \boldsymbol{\theta})}{\pi(A_t|S_t, \boldsymbol{\theta})}.$$

REINFORCE Properties

- On-policy method
- Uses the complete return from time t, which includes all future rewards until the end of the episode
- REINFORCE is thus a Monte Carlo algorithm and is only well-defined for the episodic case with all updates made in retrospect after the episode is completed
- Note that the gradient of log x is the gradient of x divided by x by the chain rule

REINFORCE Algorithm

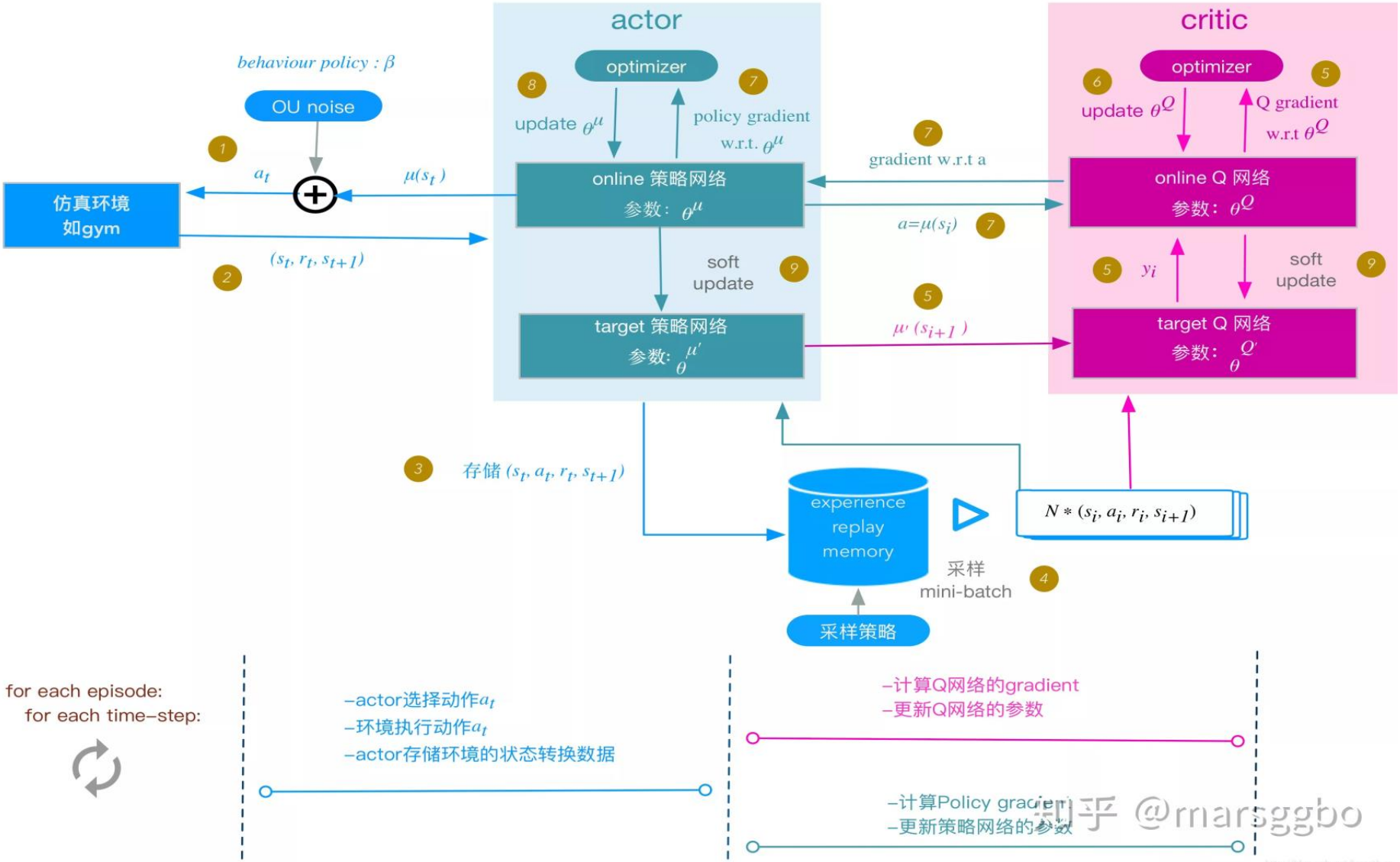
REINFORCE, A Monte-Carlo Policy-Gradient Method (episodic)

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta}), \forall a \in \mathcal{A}, s \in \mathcal{S}, \boldsymbol{\theta} \in \mathbb{R}^n$
Initialize policy weights $\boldsymbol{\theta}$
Repeat forever:
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  For each step of the episode $t = 0, \ldots, T-1$:
    $G_t \leftarrow$ return from step t
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G_t \nabla_{\boldsymbol{\theta}} \log \pi(A_t|S_t, \boldsymbol{\theta})$

Actor-Critic Methods

- Methods that learn approximations to both policy and value functions are called actor-critic methods
- Actor refers to the learned policy
- Critic refers to the learned value function, which is usually a state-value function
- The critic is bootstrapped – the state-values are updated using the estimated state-values of subsequent states
- The number of steps in the actor-critic method controls the degree of bootstrapping

Actor-Critic Diagram



知乎 @marsggbo

One-step Actor-Critic Update Rules

- On-policy method
- The state-value function update rule is the TD(0) update rule
- The policy function update rule is shown below.
- For n-step Actor-Critic, simply replace $G_t^{(1)}$ with $G_t^{(n)}$

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\doteq \boldsymbol{\theta}_t + \alpha\left(G_t^{(1)} - \hat{v}(S_t,\mathbf{w})\right)\frac{\nabla_{\boldsymbol{\theta}}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})} \\ &= \boldsymbol{\theta}_t + \alpha\left(R_{t+1} + \gamma\hat{v}(S_{t+1},\mathbf{w}) - \hat{v}(S_t,\mathbf{w})\right)\frac{\nabla_{\boldsymbol{\theta}}\pi(A_t|S_t,\boldsymbol{\theta})}{\pi(A_t|S_t,\boldsymbol{\theta})}.\end{aligned}$$

One-step Actor-Critic Algorithm

One-step Actor-Critic (episodic)

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta}), \forall a \in \mathcal{A}, s \in \mathcal{S}, \boldsymbol{\theta} \in \mathbb{R}^n$
Input: a differentiable state-value parameterization $\hat{v}(s,\mathbf{w}), \forall s \in \mathcal{S}, \mathbf{w} \in \mathbb{R}^m$
Parameters: step sizes $\alpha > 0$, $\beta > 0$

Initialize policy weights $\boldsymbol{\theta}$ and state-value weights $\mathbf{w}$
Repeat forever:
  Initialize S (first state of episode)
  $I \leftarrow 1$
  While S is not terminal:
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \beta\delta\nabla_{\mathbf{w}}\hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha I\delta\nabla_{\boldsymbol{\theta}}\log\pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Outlines

- Reinforcement Learning Introduction
- Dynamic Programming Approaches
- Monte Carlo Approaches
- Temporal Difference Approaches
- Policy Gradient Approaches
- Asynchronous Reinforcement Learning

What is Asynchronous Reinforcement Learning?

- Use asynchronous gradient descent to optimize controllers
- This is useful for deep reinforcement learning where the controllers are deep neural networks, which take a long time to train
- Asynchronous gradient descent speeds up the learning process
- Can use one multi-core CPU to train deep neural networks asynchronously instead of multiple GPUs

Parallelism (1)

- Asynchronously execute multiple agents in parallel on multiple instances of the environment
- This parallelism decorrelates the agents' data into a more stationary process since at any given time-step, the agents will be experiencing a variety of different states
- This approach enables a larger spectrum of fundamental on-policy and off-policy reinforcement learning algorithms to be applied robustly and effectively using deep neural networks
- Use asynchronous actor-learners (i.e. agents). Think of each actor-learner as a thread
- Run everything on a single multi-core CPU to avoid communication costs of sending gradients and parameters

Parallelism (2)

- Multiple actor-learners running in parallel are likely to be exploring different parts of the environment
- We can explicitly use different exploration policies in each actor-learner to maximize this diversity
- By running different exploration policies in different threads, the overall changes made to the parameters by multiple actor-learners applying updates in parallel are less likely to be correlated in time than a single agent applying online updates

No Experience Replay

- No need for a replay memory. We instead rely on parallel actors employing different exploration policies to perform the stabilizing role undertaken by experience replay in the DQN training algorithm
- Since we no longer rely on experience replay for stabilizing learning, we are able to use on-policy reinforcement learning methods to train neural networks in a stable way

Asynchronous Algorithms

- Asynchronous one-step Q-learning
- Asynchronous one-step SARSA
- Asynchronous n-step Q-learning
- Asynchronous Advantage Actor-Critic (A3C)

Asynchronous one-step Q-learning

- Each thread interacts with its own copy of the environment and at each step, computes the gradient of the Q-learning loss
- Use a globally shared and slowly changing target network with parameters $\boldsymbol{\theta}^-$ when computing the Q-learning loss
- DQN uses a globally shared and slowly changing target network too – the network is updated using a mini-batch of experience tuples drawn from replay memory
- The gradients are accumulated over multiple time-steps before being applied (similar to using mini-batches), which reduces chance of multiple actor-learners overwriting each other's updates

Exploration

- Giving each thread a different exploration policy helps improve robustness and generally improves performance through better exploration
- Use epsilon-greedy exploration with epsilon periodically sampled from some distribution by each thread

Asynchronous one-step Q-learning Algorithm

Algorithm 1 Asynchronous one-step Q-learning - pseudocode for each actor-learner thread.

```
// Assume global shared θ, θ⁻, and counter T = 0.
Initialize thread step counter t ← 0
Initialize target network weights θ⁻ ← θ
Initialize network gradients dθ ← 0
Get initial state s
repeat
    Take action a with ε-greedy policy based on Q(s, a; θ)
    Receive new state s' and reward r
    y = { r                                  for terminal s'
        { r + γ max_a' Q(s', a'; θ⁻)         for non-terminal s'
    Accumulate gradients wrt θ: dθ ← dθ + ∂(y − Q(s,a;θ))²/∂θ
    s = s'
    T ← T + 1 and t ← t + 1
    if T mod I_target == 0 then
        Update the target network θ⁻ ← θ
    end if
    if t mod I_AsyncUpdate == 0 or s is terminal then
        Perform asynchronous update of θ using dθ.
        Clear gradients dθ ← 0.
    end if
until T > T_max
```

Asynchronous one-step SARSA

- This is the same algorithm as Asynchronous one-step Q-learning except the target value used for Q(s, a) is different
- One-step SARSA uses $r + \gamma\, Q(s', a'; \boldsymbol{\theta}^{-})$ as the target value

n-step Q-learning

- So far we have been using one-step Q-learning
- One-step Q-learning updates the action value Q(s, a) towards $r + \gamma \max_{a'} Q(s', a'; \boldsymbol{\theta})$, which is the one-step return
- Obtaining a reward r only directly affects the value of the state-action pair (s, a) that led to the reward
- The values of the other state-action pairs are affected only indirectly through the updated value Q(s, a)
- This can make the learning process slow since many updates are required to propagate a reward to the relevant preceding states and actions
- Use n-step returns to propagate the rewards faster

n-step Returns

- In n-step Q-learning, Q(s, a) is updated towards the n-step return:
- $r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a)$
- This results in a single reward directly affecting the values of n preceding state-action pairs
- This makes the process of propagating rewards to relevant state-action pairs potentially much more efficient
- One-step update for the last state, two-step update for the second last state, and so on until n-step update for the nth last state
- Accumulated updates are applied in a single step

Asynchronous n-step Q-learning Algorithm

Algorithm S2 Asynchronous n-step Q-learning - pseudocode for each actor-learner thread.

```
// Assume global shared parameter vector θ.
// Assume global shared target parameter vector θ⁻.
// Assume global shared counter T = 0.
Initialize thread step counter t ← 1
Initialize target network parameters θ⁻ ← θ
Initialize thread-specific parameters θ' = θ
Initialize network gradients dθ ← 0
repeat
    Clear gradients dθ ← 0
    Synchronize thread-specific parameters θ' = θ
    t_start = t
    Get state s_t
    repeat
        Take action a_t according to the ε-greedy policy based on Q(s_t, a; θ')
        Receive reward r_t and new state s_{t+1}
        t ← t + 1
        T ← T + 1
    until terminal s_t or t − t_start == t_max
    R = { 0                    for terminal s_t
        { max_a Q(s_t, a; θ⁻)  for non-terminal s_t
    for i ∈ {t − 1, ..., t_start} do
        R ← r_i + γR
        Accumulate gradients wrt θ': dθ ← dθ + ∂(R − Q(s_i, a_i; θ'))² / ∂θ'
    end for
    Perform asynchronous update of θ using dθ.
    if T mod I_target == 0 then
        θ⁻ ← θ
    end if
until T > T_max
```

A3C

- Maintains a policy $\pi(a_t | s_t; \boldsymbol{\theta})$ and an estimate of the value function $V(s_t; \boldsymbol{\theta}_v)$
- n-step Actor-Critic method
- As with value-based methods, use parallel actor-learners and accumulate updates to improve training stability
- In practice, the policy approximation and the value function share some parameters
- Use a neural network that has one softmax output for the policy $\pi(a_t | s_t; \boldsymbol{\theta})$ and one linear output for the value function $V(s_t; \boldsymbol{\theta}_v)$ with all non-output parameters shared

Advantage Definition

- For a one-step actor-critic method with a learned policy and a learned value function, the advantage is defined as follows:
- $A(a_t, s_t) = Q(a_t, s_t) - V(s_t)$
- The quantity $r_t - V(s_t; \boldsymbol{\theta}_v)$ is an estimate of the advantage
- This estimate is used to scale the policy gradient in an actor-critic method
- The advantage estimate for a n-step actor-critic method is shown below:

$$\sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$$

A3C Algorithm

Algorithm S3 Asynchronous advantage actor-critic - pseudocode for each actor-learner thread.

// Assume global shared parameter vectors θ and θ_v and global shared counter $T = 0$
// Assume thread-specific parameter vectors θ' and θ'_v
Initialize thread step counter $t \leftarrow 1$
repeat
  Reset gradients: $d\theta \leftarrow 0$ and $d\theta_v \leftarrow 0$.
  Synchronize thread-specific parameters $\theta' = \theta$ and $\theta'_v = \theta_v$
  $t_{start} = t$
  Get state s_t
  repeat
    Perform a_t according to policy $\pi(a_t|s_t;\theta')$
    Receive reward r_t and new state s_{t+1}
    $t \leftarrow t + 1$
    $T \leftarrow T + 1$
  until terminal s_t **or** $t - t_{start} == t_{max}$
  $R = \begin{cases} 0 & \text{for terminal } s_t \\ V(s_t, \theta'_v) & \text{for non-terminal } s_t \text{// Bootstrap from last state} \end{cases}$
  for $i \in \{t-1, \ldots, t_{start}\}$ **do**
    $R \leftarrow r_i + \gamma R$
    Accumulate gradients wrt θ': $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i;\theta')(R - V(s_i;\theta'_v))$
    Accumulate gradients wrt θ'_v: $d\theta_v \leftarrow d\theta_v + \partial\,(R - V(s_i;\theta'_v))^2/\partial\theta'_v$
  end for
  Perform asynchronous update of θ using $d\theta$ and of θ_v using $d\theta_v$.
until $T > T_{max}$

Summary of Asynchronous RL

- Get faster training because of parallelism
- Can use on-policy reinforcement learning methods
- Diversity in exploration can lead to better performance than synchronous methods
- In practice, the on-policy A3C algorithm appears to be the best performing asynchronous reinforcement learning method in terms of performance and training speed

Summary

- **Reinforcement learning**
  use when need to make decisions in uncertain environment

- **Solution methods**
  - ✓ dynamic programming
    need complete model
  - ✓ Monte Carlo
  - ✓ time-difference learning (SARSA, Q-learning)
  - ✓ Approximation functions (Deep Q-learning, PG, actor-critic)
  - ✓ Asynchronous learning (AQ-learning, ASARSA, A3C)

- **Most work**
  simple algorithms
  need to design features, state representation, rewards

(PO)MDP

RL Algorithms Landscape

Policy Optimization

Dynamic Programming

modified policy iteration

DFO / Evolution

cross-entropy method

CMA

NES

Policy Gradients

REINFORCE

SVG

TRPO

PPO

Policy Iteration

Value Iteration

TD

Q-Learning

FQI

DQN

DDPG

NAF

UNREAL

Actor-Critic Methods

A3C / A2C

ACKTR

Imitation

IRL

GCL

HRL

FuN

I2A

Predictron

Model-Based Methods

MPC

PILCO

local models / LQR

Planning

(monte carlo)(tree) search, minimax, etc.

Tools: GP, GMM, (D)(C)(R)NN, LSTM, ...

Notation

- **PODMP**
  Partially Observed Markov Decision Process
- **DFO**
  Derivative-Free Optimization
- **CMA**
  Derivative-Free

  need complete model
  - ✓ Monte Carlo
  - ✓ time-difference learning (SARSA, Q-learning)
  - ✓ Approximation functions (Deep Q-learning, PG, actor-critic)
  - ✓ Asynchronous learning (AQ-learning, ASARSA, A3C)

- **Most work**
  simple algorithms
  need to design features, state representation, rewards